UNIVERSIDADE ESTADUAL DE CAMPINAS
Faculdade de Engenharia Elétrica e de Computação

Felipe Augusto Oliveira dos Santos

An Adaptive Defense System for IoT Networks Using Proactive Attack Prediction and Deep Reinforcement Learning

Campinas

2025

UNIVERSIDADE ESTADUAL DE CAMPINAS
Faculdade de Engenharia Elétrica e de Computação

Felipe Augusto Oliveira dos Santos

An Adaptive Defense System for IoT Networks Using Proactive Attack Prediction and Deep Reinforcement Learning

Dissertação apresentada à Faculdade de Engenharia Elétrica e de Computação da Universidade Estadual de Campinas como parte dos requisitos exigidos para a obtenção do título de Mestre em Engenharia Elétrica, na Área de Engenharia de Computação.

Orientador: Prof. Dr. Denis Fantinato

Este exemplar corresponde à versão final da tese defendida pelo aluno Felipe Augusto Oliveira dos Santos, e orientada pelo Prof. Dr. Denis Fantinato

Campinas

2025

Inclua aqui o pdf com a ficha catalográfica fornecida pela BAE.

Inclua aqui a folha de assinaturas.

To my family.

Agradecimentos

I would like to thank my advisor, Prof. Dr. Denis Fantinato, for his continuous guidance, patience, and insight throughout this work. I am also grateful to the Faculdade de Engenharia Elétrica e de Computação (FEEC) at Unicamp for providing the environment in which this research was carried out, and to my colleagues and family for their unwavering support.

Resumo

A proliferação exponencial da Internet das Coisas (IoT) tem produzido uma vasta rede de dispositivos interconectados, enquanto o impacto econômico global das ameaças cibernéticas continua a crescer. As restrições computacionais e de memória de muitos dispositivos IoT tornam ineficazes as soluções de segurança tradicionais e motivam mecanismos de defesa adaptativos capazes de operar autonomamente. Esta dissertação propõe e avalia rigorosamente um arcabouço de defesa adaptativo para redes IoT que combina predição proativa de ataques com Aprendizado por Reforço Profundo (DRL). O sistema é avaliado no *dataset* CICIoT2023, projetado sobre uma cadeia de eliminação cibernética (Cyber Kill Chain) de cinco estágios, com um gerador LSTM, um detector de estágio supervisionado e um ambiente Gymnasium customizado expondo um agente defensor com cinco ações. Três algoritmos DRL (DQN, PPO, A2C) são treinados ao longo de dez sementes e comparados a quatro políticas de referência e a um oráculo. O melhor agente implantável (PPO) captura 81 % do teto do oráculo com uma vantagem de latência de 141× sobre a baseline supervisionada mais forte, e todos os resultados são reprodutíveis por uma cadeia de *hashes* SHA-256 verificada de ponta a ponta.

Palavras-chaves: Segurança IoT; Aprendizado por Reforço Profundo; Cyber Kill Chain; Predição de Estágio de Ataque; LSTM; CICIoT2023; Reprodutibilidade.

Abstract

The exponential proliferation of the Internet of Things (IoT) has produced a vast network of interconnected devices while the global economic impact of cyber threats continues to grow. The computational and memory constraints of many IoT devices render traditional security solutions ineffective and motivate intelligent, adaptive defence mechanisms capable of operating autonomously. This dissertation designs, implements, and rigorously evaluates an adaptive defence framework for IoT networks that combines proactive attack-stage prediction with Deep Reinforcement Learning (DRL). The framework is evaluated end-to-end on the CICIoT2023 dataset, projected onto a five-stage Cyber Kill Chain, with an LSTM red-team generator, a supervised stage detector, and a custom Gymnasium environment exposing a five-action defender. Three DRL algorithms (DQN, PPO, A2C) are trained over ten seeds and benchmarked against four reference policies and an oracle. The best deployable agent (PPO) captures 81 % of the oracle ceiling at a 141× latency advantage over the strongest supervised baseline, and every reported number is reproducible from a SHA-256 hash chain verified end-to-end.

Keywords: IoT security; Deep Reinforcement Learning; Cyber Kill Chain; Attack Stage Prediction; LSTM; CICIoT2023; Reproducibility.

Lista de ilustrações

Lista de tabelas

Sumário

1 Introduction

1.1 The Case for Adaptive IoT Security

The Internet of Things (IoT) connects billions of embedded devices — smart cameras, industrial sensors, medical monitors, and home appliances — into a single, globally addressable network. The number of IoT devices worldwide is forecast to more than double from 19.8 billion in 2025 to over 40.6 billion by 2034 (VAILSHERY, 2025), an exponential growth that has produced a vast and multifaceted attack surface. The financial ramifications are staggering: global damages from cybercrime are projected to reach USD 12.2 trillion annually by 2031 (MORGAN, 2025). Malicious actors increasingly exploit insecure devices to steal data, disrupt services, or form large-scale botnets for Distributed Denial of Service (DDoS) attacks (NETO *et al.*, 2023; TAWALBEH *et al.*, 2020).

Securing this landscape presents unique and formidable challenges that render traditional IT security paradigms insufficient. First, most IoT devices are severely **resource-constrained**, possessing minimal computational power, memory, and energy, which precludes the use of intensive security software like traditional firewalls or antivirus agents (CHEN *et al.*, 2021). Second, IoT ecosystems are defined by extreme **heterogeneity**, comprising countless devices using disparate protocols and operating systems, which makes uniform security policy enforcement nearly impossible (MAVROEIDIS, 2023). This complexity is amplified by the large-scale, dynamic nature of these networks and the physical vulnerability of devices often deployed in unsecured locations (AL-GARADI *et al.*, 2020).

Consequently, conventional security tools are fundamentally ill-equipped for this environment. **Signature-based Intrusion Detection Systems (IDS)**, the cornerstone of many legacy systems, rely on static databases of known attack patterns. This approach is purely reactive and fails against novel or **zero-day attacks** — threats for which no signature yet exists (KHRAISAT *et al.*, 2019; YANG *et al.*, 2024). While **anomaly-based IDS** attempt to overcome this by baselining "normal" behaviour, the dynamic and heterogeneous nature of IoT traffic makes establishing a reliable baseline exceptionally difficult, leading to high **false positive rates** that can disrupt critical functions (IBITOYE *et al.*, 2021; THAREWAL *et al.*, 2022).

The core limitation of these paradigms is their lack of adaptability. They are built on static models and cannot autonomously evolve. This creates a critical need for a new security paradigm: one that is forward-looking, data-driven, and capable of auto-

nomous decision-making. **Deep Reinforcement Learning (DRL)**, a field of machine learning focused on goal-oriented learning from interaction, has emerged as a powerful framework for such a system (CHEN *et al.*, 2021). A DRL agent learns an optimal decision-making strategy, or **policy**, through trial-and-error interaction with its environment. Unlike static systems, a DRL agent can generalise from its experience to respond to novel threats and can adapt its policy as new adversarial tactics emerge, making it well-suited for the non-stationary nature of IoT security (SUTTON; BARTO, 2018; UPRETY; RAWAT, 2021). By framing cyber-defense as a sequential decision-making problem, DRL provides a robust mathematical foundation for creating autonomous security systems that can learn to outmaneuver attackers (GUERIANI *et al.*, 2023).

1.2 Problem Statement and Contributions

The preceding analysis culminates in a clear research problem: existing IoT security frameworks are not simultaneously **proactive**, **adaptive**, **data-driven**, and **rigorously benchmarked** against deployable reference policies. Reactive signature-based systems cannot anticipate novel attacks, supervised classifiers do not act, and prior DRL-for-IoT work seldom reports head-to-head comparisons against the deployable baselines a security operator actually has at their disposal — random, always-block, and a supervised stage classifier composed with a recommended-action mapping — under a unified evaluation contract.

This dissertation directly addresses that gap by designing, implementing, and rigorously evaluating a closed-loop adaptive defense framework for IoT networks that combines proactive attack prediction with Deep Reinforcement Learning, validated end-to-end on the contemporary CICIoT2023 dataset. Concretely, this work makes the following contributions:

1. **A kill-chain-aware proactive-adaptive defense framework.** A two-stage architecture combining (i) a Long Short-Term Memory (LSTM) stage detector and an LSTM red-team episode generator trained on stage-labelled CICIoT2023 sequences with (ii) a custom Gymnasium environment that exposes a five-action defender (observe, log, throttle, block, isolate) under a stage-action proportionality reward inspired by the IoTWarden recommended-action mapping (ALAM *et al.*, 2024). The attacker model is explicitly bounded by a **monotonic-attacker assumption**: the red-team LSTM generates upper-triangular stage transitions in which the kill chain advances but never retreats, consistent with the IoTWarden threat model and the CICIoT2023 attack taxonomy.

2. **Grounding in a realistic, modern dataset.** The framework is developed and validated on the **CICIoT2023 dataset** (NETO *et al.*, 2023), projected onto a five-stage Cyber Kill Chain (benign, recon, access, maneuver, impact) with a documented per-class mapping. A leakage bug discovered during model training (where four held-out out-of-distribution attack classes had inadvertently leaked 70 % of their rows back into the training pool) was fixed and locked behind disjointness assertions before any benchmark numbers were collected.

3. **A comparative DRL benchmark with a 141× latency advantage over the supervised deployable baseline.** DQN, PPO, and A2C are trained over ten seeds and evaluated on a held-out balanced test split against four reference policies: random, always-observe, always-block, and a supervised RandomForest stage classifier composed with the recommended-action mapping (RF-Acting). A rule-based policy that observes the true attack stage is reported as an *oracle ceiling*. The trained agents capture **81 %** of that ceiling with non-overlapping bootstrap confidence intervals against every non-RL deployable baseline. Among deployable policies the comparison is a latency–reward trade-off: trained PPO achieves inference at 0.098 ms per decision versus RF-Acting at 13.85 ms — a **141× latency advantage** — at a reward cost of approximately 10 %.

4. **Reward-component and out-of-distribution ablations.** A 12-cell sparse one-at-a-time sweep over the five reward coefficients × $\{0.5\times, 1\times, 2\times\}$ plus a binary environment-semantics flag identifies a structural change to the impact-stage termination contract that closes 71 % of the residual gap to the oracle ceiling and lifts the mitigated-impact rate from 0.153 to 0.900. A separate evaluation against four held-out out-of-distribution attack classes characterises the policy's behaviour on signature-novel inputs and yields the pre-registered finding that the trained policy is robust to, but not better at, the OOD class on which the supervised stage detector is structurally blind.

5. **An audit-first reproducibility protocol.** Every figure ships a `manifest.json` that pins the SHA-256 hashes of every input artefact (data splits, trained models, scaler, episode roll-outs) and the producing Git commit. A smoke-reproducibility harness verifies the full hash chain end-to-end on a fresh checkout in approximately five seconds. The protocol is documented in Appendix A.

The remainder of this dissertation is structured as follows. Chapter 2 provides a detailed review of background concepts in IoT security, the Cyber Kill Chain, and reinforcement learning, and situates this work against related research. Chapter 3 details the proposed framework architecture, including the kill-chain projection of CICIoT2023, the

LSTM red team and stage detector, the Markov Decision Process underlying the environment, the reward function, the training protocol, and the baseline policies. Chapter 4 presents the empirical results across all phases of the pipeline and discusses the headline findings, including the oracle-ceiling reframe, the reward-component ablation, and the out-of-distribution robustness story. Chapter 5 summarises the contributions, discusses limitations and threats to validity, and outlines future work.

2 Background and Related Work

This section establishes the theoretical groundwork and contextualises the contributions of this thesis. Section 2.1 introduces the Cyber Kill Chain abstraction adopted as the unit of analysis throughout the dissertation. Section 2.2 reviews the foundational principles of Reinforcement Learning and the specific Deep Reinforcement Learning algorithms central to this work. Section 2.6 introduces the deployable-baseline taxonomy and the role of the rule-based oracle as a measurement instrument. Section 2.7 provides a review of related research in IoT security and DRL-based defense and positions this thesis within that literature.

2.1 The Cyber Kill Chain as a Defender's Decision Frame

The **Cyber Kill Chain** (MAVROEIDIS, 2023) is a conceptual model that decomposes a sustained intrusion into a sequence of progressively more compromising stages. Originally formulated by industrial threat-intelligence practitioners and refined by the academic community, it organises adversary behaviour as an ordered chain in which earlier stages prepare for the success of later stages and where defender opportunity costs grow monotonically with stage progression: it is much cheaper to interrupt an attack at RECON than at IMPACT.

Following the formulation adopted in this thesis (and consistent with the IoTWarden recommended-action mapping (ALAM *et al.*, 2024)), traffic on an IoT network is projected onto a five-stage kill chain:

1. **benign** — routine, non-malicious traffic. The defender's correct posture is observational.
2. **recon** — the adversary scans, fingerprints, or otherwise enumerates the network and its assets. Behaviour is malicious but the network is not yet compromised; the defender's correct posture is to log and continue monitoring.
3. **access** — the adversary establishes an initial foothold (e.g. exploit-driven authentication bypass, brute-force, command injection). The defender should throttle the implicated traffic and prepare to escalate.
4. **maneuver** — the adversary performs lateral movement, privilege escalation, or other in-network manipulation to position for a final action. The defender should block.

5. **impact** — the adversary's terminal action: data exfiltration, denial-of-service, ransomware detonation. The defender's last opportunity to prevent loss is to isolate the affected segment.

The kill chain is more than narrative scaffolding: it provides a discrete decision frame in which (i) the attacker's evolving state is summarised by a single ordered variable and (ii) the defender's action menu can be aligned to that variable through a recommended-action mapping (BENIGN → OBSERVE, RECON → LOG, ACCESS → THROTTLE, MANEUVER → BLOCK, IMPACT → ISOLATE). This alignment is the basis of the stage-action proportionality reward used in Section 3.12, and it is what makes the rule-based recommended-action policy a meaningful upper bound (Section 2.6).

2.2 Reinforcement Learning for Autonomous Systems

Reinforcement Learning (RL) is a computational paradigm focused on goal-directed learning from interaction. An intelligent agent learns to make optimal decisions not by being shown labeled examples, but by exploring an environment and discovering which sequences of actions yield the greatest cumulative reward (SUTTON; BARTO, 2018). This learning process is formalised through the mathematical framework of Markov Decision Processes (MDPs).

SUB

2.3 Markov Decision Processes

An MDP provides a formal model for sequential decision-making under uncertainty. The core principle is the **Markov Property**, which asserts that the current state provides all necessary information to make an optimal decision, rendering the history of prior states irrelevant (SUTTON; BARTO, 2018). An MDP is defined as a five-tuple (S, A, P, R, γ):

- $\boldsymbol{S}$: A finite set of states, representing all possible configurations of the environment. In this thesis, a state $s \in S$ is a multi-modal observation composed of a vector of CICIoT2023 traffic features, a history of recent states and actions, and (optionally) the discrete predicted attack stage produced by the LSTM stage detector.
- $\boldsymbol{A}$: A finite set of actions. For this work, A is the five-action defender menu derived from the kill chain in Section 2.1: $A = \{\text{OBSERVE}, \text{LOG}, \text{THROTTLE}, \text{BLOCK}, \text{ISOLATE}\}$, ordered by escalating defensive force and operational disruption.

- $\boldsymbol{P}$: The state transition probability function, $P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$, defining the dynamics of the environment.
- $\boldsymbol{R}$: The reward function, $R(s, a, s')$, which provides the immediate numerical feedback for transitioning from s to s' via action a. The full as-built reward used in this thesis is detailed in Section 3.12.
- $\boldsymbol{\gamma}$: The discount factor ($0 \leq \gamma \leq 1$), which prioritises immediate rewards over future ones.

The agent's objective is to learn a **policy**, $\pi(a \mid s)$, which maps states to a probability distribution over actions, to maximise the expected discounted return.

SUB

2.4 Value Functions and Bellman Equations

To determine the quality of states and actions, RL uses value functions. The **state-value function**, $V^\pi(s)$, is the expected return from state s while following policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s \right] \tag{2.1}$$

The **action-value function** (or Q-function), $Q^\pi(s, a)$, is the expected return after taking action a from state s and then following policy π:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s, A_t = a \right] \tag{2.2}$$

These functions adhere to the recursive relationships defined by the Bellman equations. The **Bellman optimality equation** for the Q-function is central to many RL algorithms and is expressed as:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[R(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \tag{2.3}$$

Solving this equation yields the optimal Q-function, $Q^*(s, a)$, from which the optimal policy can be derived by selecting the action with the highest value in each state.

SUB

2.5 Deep Reinforcement Learning Algorithms

For environments with large or continuous state spaces, like an IoT network, tabular methods are intractable. Deep Reinforcement Learning (DRL) overcomes this

by using deep neural networks to approximate the value functions or policies (CHEN *et al.*, 2021). This thesis implements and benchmarks three seminal DRL algorithms that represent the primary families of model-free RL.

2.5.0.0.1 Deep Q-Networks (DQN).

DQN is a value-based, off-policy algorithm that uses a neural network to approximate the action-value function, $Q(s, a; \theta) \approx Q^*(s, a)$ (MNIH *et al.*, 2015). To stabilise learning, DQN introduces two critical mechanisms:

- **Experience Replay:** Experiences (s_t, a_t, r_t, s_{t+1}) are stored in a large replay buffer. The network is trained on mini-batches randomly sampled from this buffer, which breaks temporal correlations in the data.
- **Fixed Target Network:** A separate, periodically updated target network is used to generate stable target values for the Bellman equation, preventing oscillations during training. The loss function is the Mean Squared Bellman Error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta^-)}_{\text{Target Q-value}} - \underbrace{Q(s, a; \theta)}_{\text{Predicted Q-value}}\right)^2\right] \tag{2.4}$$

  where θ^- are the parameters of the fixed target network and $\mathcal{D}$ is the replay buffer.

2.5.0.0.2 Proximal Policy Optimization (PPO).

PPO is a state-of-the-art on-policy actor-critic algorithm that directly optimises a parameterised policy (the actor) while using a value function (the critic) to reduce variance (SCHULMAN *et al.*, 2017). Its key innovation is a clipped surrogate objective function that constrains the size of policy updates, ensuring greater training stability:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right] \tag{2.5}$$

where $r_t(\theta)$ is the probability ratio between the new and old policies, $\hat{A}_t$ is the estimated advantage, and ϵ is a hyperparameter that constrains the policy change. This approach prevents destructive policy updates and is known for its reliability and performance.

2.5.0.0.3 Advantage Actor-Critic (A2C).

A2C is a synchronous variant of the foundational Asynchronous Advantage Actor-Critic (A3C) algorithm (MNIH *et al.*, 2016). Like PPO, it is an actor-critic method,

but it uses a simpler, unconstrained policy update mechanism. It typically employs multiple parallel workers to collect experience simultaneously; these experiences are then aggregated to compute a single stable gradient update for both the actor (policy) and the critic (value function). While often faster to converge than PPO on simpler problems, it can be less stable.

All three algorithms are implemented in this thesis using Stable Baselines3 `MultiInputPolicy`, which is designed to handle the dictionary-shaped observation space exposed by the custom IoT environment described in Section 3.12.

2.6 Deployable Baselines and the Role of an Oracle Reference

A defining methodological choice of this work is to evaluate the trained DRL agents not only against *trivial* baselines (random, always-OBSERVE, always-BLOCK) but also against two stronger references that bracket the achievable performance under the kill-chain decision frame:

- **RF-Acting (deployable supervised + rule).** A composite policy that uses a supervised RandomForest classifier (trained in the stage-detection phase) to predict the current attack stage from the observed feature vector, then maps the predicted stage to an action via the recommended-action mapping. RF-Acting is fully deployable: it consumes only what a real defender can observe and produces a defensive action at every step.
- **Recommended-Action rule (oracle reference, *not* a competing baseline).** An identical action-selection rule that, instead of consuming a learned classifier's output, reads the *true* attack stage directly from the simulator's privileged information. This oracle is therefore a measurement instrument that quantifies the value of perfect stage detection under the chosen reward function. It is reported as an *upper bound* on what any deployable policy could achieve under the recommended-action mapping; it is *not* reported as a method this work claims to outperform, because the oracle does not run on observable inputs and cannot be deployed.

The trained DRL agents are deployable in the same sense as RF-Acting — they observe a vector of CICIoT2023 traffic features and (optionally) the predicted stage, and produce an action — and are scored against both deployable baselines and against the oracle ceiling. Reporting the oracle as a measurement instrument rather than a competing baseline is a methodological commitment that frames the headline thesis claim as "trained RL captures $X\,\%$ of the value of perfect stage detection" rather than the weaker and

potentially misleading "trained RL beats the recommended-action policy". This choice was retroactively endorsed by the audit-first review protocol in this work and is documented in detail in Section 4.

2.7 Related Work in AI-Driven IoT Security

The application of artificial intelligence to IoT security is an active and rapidly evolving field of research. Early work focused on applying classical machine learning models to intrusion detection, which has since paved the way for more sophisticated, adaptive systems based on Deep Reinforcement Learning.

SUB

2.8 Machine Learning Approaches for Intrusion Detection

Early approaches to intelligent IoT security leveraged supervised and unsupervised machine learning for building Intrusion Detection Systems (IDS). Supervised models, such as Support Vector Machines and Random Forests, were trained on labeled datasets to classify network flows as either benign or malicious (AL-GARADI *et al.*, 2020). While effective at identifying known attack patterns, these models struggle with novel or **zero-day attacks** not seen during training, a limitation they share with traditional signature-based IDS (KHRAISAT *et al.*, 2019). Furthermore, they typically operate as passive classifiers, identifying threats without the capacity for automated, sequential defensive actions — a limitation this thesis addresses directly by composing the supervised classifier with a recommended-action rule (RF-Acting) and using it as a deployable baseline against which the DRL agents are scored.

SUB

2.9 The Shift to Active Defense with DRL

The limitations of static ML models motivated a shift towards DRL to create autonomous agents capable of active, closed-loop cyber-defense. Several notable works have explored this domain and provide the foundation upon which this thesis builds.

Some research has focused on using RL for deception and intelligence gathering. The work of Guan et al. in **HoneyIoT** (GUAN *et al.*, 2023) presents an adaptive high-interaction honeypot that uses RL to learn an attacker's behavior and dynamically alter its services to keep the attacker engaged and reveal their methods. While this is a

powerful application of RL, its primary purpose is reconnaissance, whereas the system presented in this thesis is designed as a direct defense mechanism for a live network.

Other works have targeted specific layers of the IoT ecosystem. **IoTWarden**, proposed by Alam et al. (ALAM *et al.*, 2024), uses a DQN agent to mitigate attacks in trigger-action IoT platforms (e.g. IFTTT) by identifying and disabling malicious rules or "applets" in real time. This thesis adopts the IoTWarden *recommended-action mapping* (BENIGN → OBSERVE, RECON → LOG, . . . , IMPACT → ISOLATE) as a design choice for the environment's reward function, and treats IoTWarden as an inspiration rather than a head-to-head baseline: IoTWarden was evaluated on a different (synthetic, trigger-action) environment with a different action space, so direct numerical comparison is not methodologically sound. The headline numerical claim of this thesis is therefore framed against the in-domain oracle ceiling described in Section 2.6, never against an external IoTWarden number.

The work by Nguyen et al. (NGUYEN *et al.*, 2021) proposes a Federated DRL system for traffic monitoring in Software-Defined Networks for IoT, focusing on efficient data collection and analysis rather than active defense. Tharewal et al. (THAREWAL *et al.*, 2022) apply DRL to industrial IoT intrusion detection but treat the agent as a classifier rather than a sequential decision-maker. The research in this thesis addresses a more fundamental challenge by operating at the **network layer**, analysing features derived from raw traffic to defend against a broad range of network-level attacks such as DDoS, DoS, and Spoofing, which are prevalent in the CICIoT2023 dataset, and by structuring the policy as a kill-chain-aware sequential decision-maker.

SUB

2.10 Comparison of Closely Related DRL-Based IoT Defense Systems

Table 2.1 positions the three most closely related prior works — IoTWarden (ALAM *et al.*, 2024), HoneyIoT (GUAN *et al.*, 2023), and Nguyen et al. (NGUYEN *et al.*, 2021) — against this thesis along the key dimensions that determine deployability and empirical rigour.

The key differentiators are (i) training and evaluation on a large-scale, real-capture dataset rather than a synthetic or simulated environment; (ii) an explicit oracle-ceiling measurement instrument that quantifies the value of perfect stage detection; and (iii) head-to-head reward comparisons against deployable baselines under a unified evaluation contract.

Tabela 2.1 – Comparison of DRL-based IoT security systems. "Deployable" means the policy runs on observable network inputs without simulator-privileged state. "Head-to-head benchmark" means the paper reports numerical comparisons against at least one deployable non-RL baseline under a shared evaluation contract.

System	Environment	Action space	Dataset	Deployable?	Head-to-head benchmark?
IoTWarden (ALAM *et al.*, 2024)	Synthetic trigger-action (IFTTT applets)	Disable/enable applet rules	Synthetic rule logs	Yes	No (no non-RL baseline)
HoneyIoT (GUAN *et al.*, 2023)	Simulated honeypot	Alter service offerings	Synthetic attacker model	Reconnaissance only	No (no reward comparison)
Nguyen et al. (NGUYEN *et al.*, 2021)	Software-Defined Network (IoT)	Traffic monitoring actions	Custom SDN traces	Passive classifier	No (accuracy only)
This work	**Real CICIoT2023 traffic (105 IoT devices)**	**5-action kill-chain defender**	**CICIoT2023 (NETO *et al.*, 2023)**	**Yes**	**Yes (oracle, RF-Acting, 3 trivials; 10 seeds; bootstrap CIs)**

SUB

2.11 Positioning the Thesis: Bridging the Research Gap

While previous works have successfully applied DRL to specific aspects of IoT security, a gap remains for a comprehensive, network-level defense system that is simultaneously:

- **Proactive and Adaptive:** combining a supervised stage detector and an LSTM-based red-team episode generator with a DRL-based agent in a closed loop from threat anticipation to autonomous action.
- **Grounded in Realistic, Modern Data:** validated on the large-scale and contemporary **CICIoT2023 dataset**, with documented kill-chain projections and a fully

audited splits protocol that closes a subtle out-of-distribution leakage bug found during model training.

- **Network-Layer Focused:** designed to defend against a broad range of fundamental network attacks by analysing traffic-flow features.
- **Rigorously Benchmarked:** providing a thorough empirical comparison of modern value-based (DQN) and actor-critic (PPO, A2C) algorithms against three trivial baselines, a deployable supervised + rule baseline (RF-Acting), and an oracle reference — with non-overlapping bootstrap confidence intervals as the statistical-separation criterion.
- **Reproducible by Construction:** every figure ships with a SHA-256 hash chain over its inputs and a smoke-reproducibility harness verifies the chain end-to-end on a fresh checkout.

This thesis directly addresses these gaps by designing, implementing, and evaluating a complete framework that integrates these critical components, thereby advancing the development of resilient, intelligent systems capable of defending the vast and ever-expanding IoT landscape.

3 Methodology

This section details the methodology used to design, implement, and evaluate the proposed adaptive IoT defense system. The framework is a multi-stage closed-loop pipeline that synergises predictive modelling with autonomous decision-making to produce a proactive security posture. Section 3.1 opens with an explicit threat model and attacker-model boundary. Section 3.5 presents the high-level architecture. Section 3.6 describes the CICIoT2023 dataset and the kill-chain projection that turns per-flow labels into episode trajectories. Section 3.10 introduces the LSTM-based red-team episode generator, and Section 3.11 the supervised stage-detection module. Section 3.12 formally specifies the Markov Decision Process underlying the custom Gymnasium environment, including the as-built reward function. Section 3.19 details the blue-team training protocol, and Section 3.20 defines the deployable baselines and the oracle reference. Section 3.21 describes the reward-component and out-of-distribution ablations. Section 3.25 introduces the audit-first reproducibility framework that pins every figure to the SHA-256 hash chain of its inputs.

3.1 Threat Model and Attacker-Model Boundary

SUB

3.2 Attacker Capabilities and Objectives

The framework defends against a **network-layer attacker** that targets one or more IoT devices in a monitored segment. The attacker is capable of executing all 33 attack types represented in the CICIoT2023 dataset, organised into seven categories: Distributed Denial of Service (DDoS), Denial of Service (DoS), Reconnaissance, Web-based exploitation, Brute Force, Spoofing, and Mirai botnet variants. This coverage includes attack patterns mapped to the reconnaissance, exploitation, lateral movement, and impact stages of the MITRE ATT&CK framework (MAVROEIDIS, 2023); specifically, CICIoT2023 Reconnaissance classes correspond to MITRE ATT&CK *Reconnaissance* (TA0043) and *Discovery* (TA0007); Access classes to *Initial Access* (TA0001) and *Credential Access* (TA0006); Maneuver classes to *Lateral Movement* (TA0008) and *Defense Evasion* (TA0005); and Impact classes to *Impact* (TA0040).

SUB

3.3 Monotonic-Attacker Assumption

A central simplifying assumption of this work is the **monotonic-attacker model**: the attacker's kill-chain stage advances forward along the sequence BENIGN → RECON → ACCESS → MANEUVER → IMPACT but never retreats to an earlier stage absent defender intervention. This is encoded in the red-team LSTM via an upper-triangular transition mask (Section 3.10) and is consistent with the IoTWarden threat model (ALAM *et al.*, 2024).

This assumption is a deliberate scope boundary, not a claim about all real-world attackers. It holds for the dominant attack patterns in CICIoT2023 (sequential DDoS, brute-force, and Mirai-family chains) but excludes adversaries that interleave or repeat stages, pivot between targets, or use multi-wave campaigns. The implications for the defender are two-fold: (i) de-escalation bonuses in the reward function are well-defined only under monotonicity; and (ii) the threat model does *not* capture advanced persistent threat (APT) scenarios with non-linear kill chains. Relaxing the monotonic assumption to a non-monotonic attacker model is identified as future work (Section 5.4).

SUB

3.4 Defender Observability

The defender observes a 29-dimensional per-flow feature vector derived from the CICIoT2023 traffic capture. It does *not* observe the true kill-chain stage; that is privileged simulator information accessible only to the oracle reference policy. The defender may optionally consume the output of a frozen supervised stage classifier (used by the RF-Acting baseline and in evaluation; not used during RL training). This observability contract is fixed throughout the dissertation and is the basis for the "81% of the oracle ceiling" headline claim.

3.5 Framework Architecture Overview

The proposed framework is designed to transition IoT security from a reactive to a proactive paradigm. This is achieved through the architecture illustrated in Figure 3.1, which can be understood as a closed-loop pipeline of two offline preparation stages and one online learning loop.

The pipeline has three stages:

1. **Stage I — Predictive Foundation (offline).** The CICIoT2023 dataset is pro-

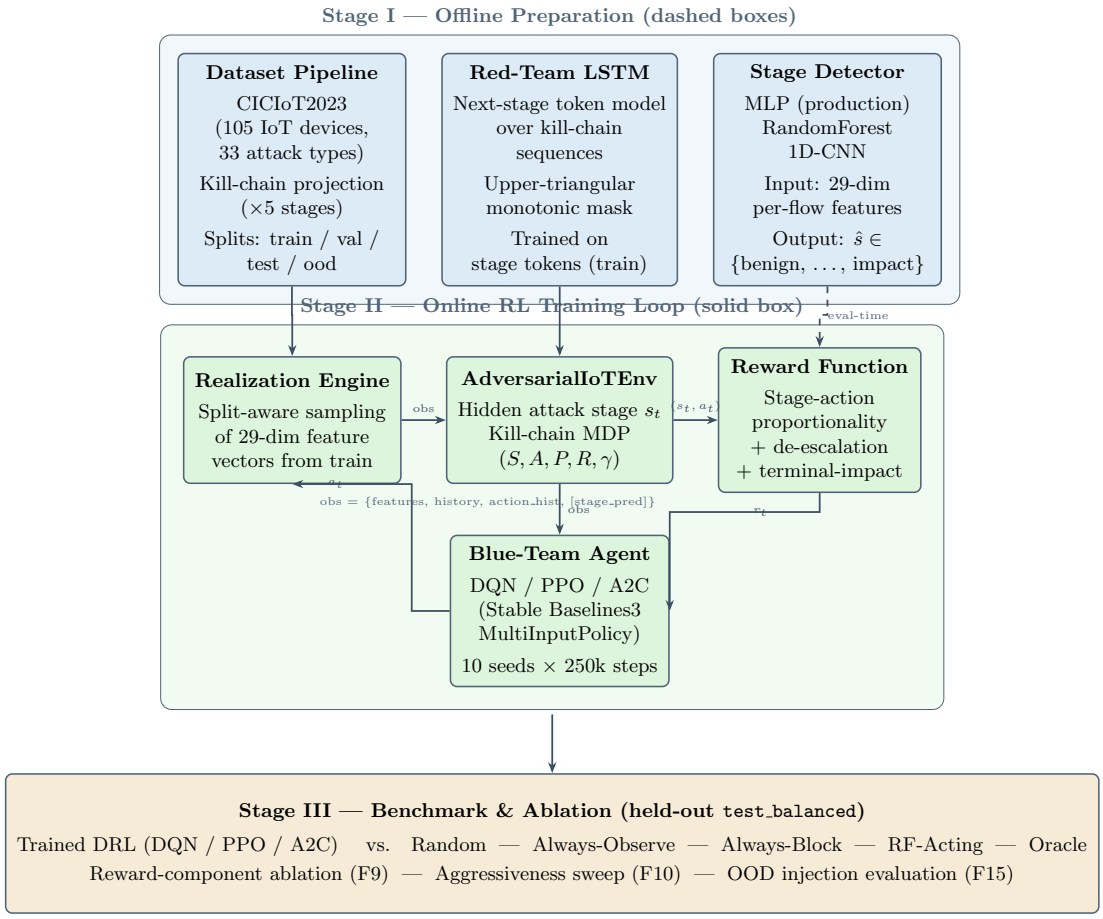


Figura 3.1 – High-level architecture of the proactive-adaptive defense framework. **Stage I (offline, dashed background):** three preparation lanes — Dataset Pipeline (CICIoT2023 kill-chain projection and split construction), Red-Team LSTM (next-stage token model with upper-triangular monotonic mask), and Supervised Stage Detector (MLP / RandomForest / 1D-CNN). **Stage II (online, solid background):** the closed-loop RL training environment comprising the Realization Engine (split-aware feature sampling), AdversarialIoTEnv (kill-chain MDP), stage-action reward function, and the Blue-Team Agent (DQN/PPO/A2C, 10 seeds × 250k steps). The dashed arrow from the Stage Detector carries the predicted stage $\hat{s}$ used only at evaluation time. **Stage III:** held-out benchmark against deployable baselines and the oracle reference, plus reward-component and OOD-robustness ablations.

jected onto a five-stage kill chain (Section 3.6). Two LSTM-based components are trained on the resulting stage-labelled splits: a *red-team episode generator* that samples next-stage transitions for episode roll-outs (Section 3.10), and a *stage detector* that classifies a per-flow feature vector into one of the five kill-chain stages (Section 3.11). A supervised RandomForest classifier is also trained as a deployable baseline (Section 3.20).

2. **Stage II — Adaptive Defense System (online training).** A custom Gymnasium environment instantiates the kill-chain MDP (Section 3.12); the red-team LSTM drives stage transitions and a split-aware feature realiser samples observed feature vectors from the appropriate dataset partition. Three DRL algorithms (DQN, PPO, A2C) are trained against this environment (Section 3.19).

3. **Stage III — Benchmarking and Ablation.** The trained checkpoints are evaluated on the held-out `test_balanced` split against four deployable baselines and an oracle reference (Section 3.20); reward-component and out-of-distribution ablations characterise the policy's sensitivity to environment design (Section 3.21).

The pipeline is decoupled by development stage, with each stage emitting an immutable record under `docs/results/<phase>/` that pins the SHA-256 hash of every input artefact (Section 3.25). This discipline allows the empirical record to be re-verified end-to-end on a fresh checkout in seconds, without retraining any model.

3.6 Data Source and Kill-Chain Projection

The empirical validity of any AI-driven security system depends critically on the quality and realism of its training data. This research uses the **CICIoT2023 dataset** (NETO *et al.*, 2023), a contemporary and extensive resource for IoT security research that addresses the limitations of older synthetic benchmarks.

SUB

3.7 The CICIoT2023 Dataset

CICIoT2023 was developed by the Canadian Institute for Cybersecurity to provide a realistic benchmark reflecting modern IoT network traffic. Its key characteristics are:

- **Realistic testbed:** traffic was captured from a physical topology of 105 real IoT devices, including smart cameras, speakers, sensors, and hubs (NETO *et al.*, 2023).

- **Comprehensive attack coverage:** the dataset contains benign traffic alongside labelled data for 33 distinct attack types, grouped into seven major categories: DDoS, DoS, Reconnaissance, Web-based, Brute Force, Spoofing, and Mirai variants.
- **Modern attack vectors:** attacks were executed *by* other compromised IoT devices within the testbed, simulating realistic botnet and device-to-device attack scenarios.

SUB

3.8 Kill-Chain Projection

A per-class mapping projects each of the 34 CICIoT2023 labels (33 attack classes plus benign) onto exactly one of the five kill-chain stages (BENIGN, RECON, ACCESS, MANEUVER, IMPACT) introduced in Section 2.1. The full table is provided in Appendix B; the design principle is that classes describing scanning or fingerprinting (e.g. `Recon-OSScan`, `Recon-PortScan`) map to RECON, classes describing exploit-driven foothold establishment (`BrowserHijacking`, `CommandInjection`, brute-force variants) map to ACCESS, lateral-movement and spoofing classes map to MANEUVER, and direct denial-of-service or data-loss classes (most DDoS, DoS, and Mirai variants) map to IMPACT. The kill-chain stage is the only label the downstream RL pipeline consumes; the original 34-class label is retained only for the supervised stage-detector evaluation.

SUB

3.9 Splits Protocol and Out-of-Distribution Reservation

Four attack classes are reserved as held-out *out-of-distribution* (OOD) classes that the supervised models and RL agents never see during training: `DDoS-HTTP_Flood` (IMPACT), `Mirai-udpplain` (IMPACT), `XSS` (ACCESS), and `VulnerabilityScan` (RECON). The remaining 30 classes are stratified-split into `train` (70 %), `val_balanced` (15 %), and `test_balanced` (15 %) partitions with disjoint indices. The disjointness of the four sets (`train`, `val_balanced`, `test_balanced`, `ood`) is locked behind explicit assertions in `tests/test_build_split_indices.py`.

A subtle leakage bug in the original splits-construction script was discovered during stage-detector training: the OOD classes were correctly identified as held-out but were not removed from the `train`/`val`/`test` index pools, so 70 % of every "held-out" class was simultaneously a training row. The fix (commit `3cd2fb9`) computes the OOD indices first, masks them out of the index pool, and only then performs the stratified split; the disjointness assertions above were added in the same commit. The leakage bug and its

fix are presented honestly in the results chapter (Section 4, §4.2) as a teachable example of the audit-first protocol catching a real bug before it could contaminate downstream stages.

3.10 Red-Team Episode Generator

The training-time environment requires a generative model of attacker behaviour that produces realistic stage-to-stage transitions. The red-team component is a single-layer Long Short-Term Memory (LSTM) network with hidden size 128 trained for 15 epochs with early stopping on the validation cross-entropy loss (HOCHREITER; SCHMIDHUBER, 1997). The network is conditioned on a prefix of past stages and predicts the next stage as a softmax distribution over the five kill-chain stages.

The training data are stage-token sequences derived from per-attack-class sequence priors built from the `train` split: each attack class contributes its empirical attacker behaviour to a class-conditional transition matrix, and episodes are generated by composing those priors. Consistent with the monotonic-attacker assumption (Section 3.3), the LSTM is trained with an upper-triangular transition mask: once the chain advances to a more compromising stage, it does not return absent defender intervention. Section 4, §4.2 presents the validation curves and the empirical 5×5 transition matrix the trained LSTM generates.

3.11 Supervised Stage Detector

The supervised stage detector classifies a 29-feature CICIoT2023 vector into one of the five kill-chain stages. Three architectures are trained and compared; Table 3.1 summarises their key characteristics.

Tabela 3.1 – Comparison of supervised stage-detector architectures on the held-out `test_balanced` split. "Production" indicates the model used in the RL environment's stage-prediction feature. Latency is median per-sample CPU inference on the evaluation host.

Model	**Params**	**Size (KB)**	**Macro-F1**	**Latency (ms)**	**Role**
MLP (production)	4,357	20	0.786	0.039	RL stage featu
RandomForest (100 trees)	∼2.6 M	22,000	0.904	0.039	RF-Acting bas
1D-CNN	12,800	55	0.723	0.041	Reference only

The RandomForest achieves higher in-distribution macro-F1 (0.904 vs. 0.786 for the MLP) but is approximately three orders of magnitude larger on disk and is retained primarily as the supervised classifier underlying the deployable RF-Acting baseline

(Section 3.20). The production MLP is selected for the RL environment's optional stage-prediction field on the grounds of size and latency parity with the trained RL agents. The 1D-CNN baseline is retained for completeness.

All three models are trained on the post-leakage-fix `train` split (281,420 rows after OOD reservation) and evaluated on the held-out `test_balanced` split. Per-stage recall is reported as the principal in-distribution metric and per-class recall as the principal out-of-distribution metric. The detector's behaviour on the four held-out OOD classes is the upstream input to the RL-level robustness story (Section 3.21, single-stage OOD injection evaluation): the same OOD-asymmetry pattern (one class with recall 0.001 and three classes with recall ≥ 0.79) recurs in the RL evaluation in a way that is best understood through the supervised result first.

3.12 Reinforcement Learning Environment

The RL environment is a custom Gymnasium task that instantiates the kill-chain MDP. Its core components are the state space, the five-action defender, and the as-built piecewise reward function. Together they implement the formal MDP (S, A, P, R, γ) introduced in Section 2.3.

SUB

3.13 State and Action Spaces

The environment uses a `Dict` observation space that gives the agent a multi-modal view of the network state. At each step the observation contains:

- `current_state`: the 29-dimensional CICIoT2023 traffic feature vector at the current step, sampled by a split-aware realisation engine that draws from the named partition (`train` during training; `test_balanced` during evaluation) and never from any OOD-reserved row.
- `state_history`: a w-step buffer of recent feature vectors, allowing the agent to perceive temporal trends. The window size $w = 5$ is selected as the default based on an ablation study (Section 3.18) which shows that $w = 5$ achieves peak mean reward and that increasing to $w = 10$ provides no statistically significant improvement.
- `action_history`: a buffer of the agent's most recent actions, providing self-awareness of recent behaviour.

- `stage_pred` (optional, evaluation-time only): the discrete predicted stage from a frozen supervised classifier, used by the deployable RF-Acting baseline; the trained DRL agents do not consume this field during training.

The action space is the ordered five-action defender menu introduced in Section 2.2: $A = \{\text{OBSERVE} = 0, \text{LOG} = 1, \text{THROTTLE} = 2, \text{BLOCK} = 3, \text{ISOLATE} = 4\}$. The ordering matters because the reward function below penalises actions outside a ± 1 band of the recommended action for the current stage.

SUB

3.14 Episode Lifecycle

The episode begins at $t = 0$ in the BENIGN stage. At each step, the agent observes the state and chooses an action; the environment then advances the attacker stage according to one of three rules:

1. **Defender-driven de-escalation** (probability $p_{\text{de-esc}} = 0.6$). If the agent picks BLOCK or ISOLATE on an active ACCESS or MANEUVER stage, with probability $p_{\text{de-esc}}$ the environment resets the stage to BENIGN and grants the agent a $b_{\text{success}} = +250$ bonus. The value $p_{\text{de-esc}} = 0.6$ is justified by an aggressiveness sweep (Section 3.16): at $p = 0.6$ the trained PPO policy reaches within 200 reward of the oracle ceiling, and the curve's plateau indicates this is the sweet spot where defensive agency is meaningful but not trivially achieved.

2. **Natural LSTM transition** (otherwise). The red-team LSTM samples the next stage from its conditional distribution given the recent transition history.

3. **Lifecycle-floor impact clamp.** If the natural transition produces IMPACT before $t = \texttt{min\_episode\_length} = 20$, the stage is clamped to MANEUVER for that step. This reflects the threat-model assumption that real-world IMPACT requires time-to-execute and is not an instantaneous jump from RECON.

The episode terminates when the chain reaches IMPACT (default environment contract; this can be flipped by the `impact_is_terminal` flag exercised in the F9 ablation, see Section 3.21) or is truncated at `max_steps` = 500.

SUB

3.15 Reward Function

The per-step reward at step t is a piecewise sum of **six independent reward signals**. Let a_t denote the action chosen at step t, s_t the decision-time stage (the stage the agent observed when it chose a_t, not the next stage), and $rec(s)$ the recommended-action mapping (BENIGN maps to OBSERVE, RECON to LOG, ACCESS to THROTTLE, MANEUVER to BLOCK, IMPACT to ISOLATE). The six per-step reward components are:

1. **Action cost.** A per-action cost $-\alpha \cdot C_{\text{action}}(a_t)$ is always applied, scaled by the action-cost scale α (default $\alpha = 1$).
2. **Proportionality bonus.** A bonus of $+r_{\text{prop}}$ when the chosen action is within ± 1 of the recommended action for the current stage, i.e. when $|a_t - rec(s_t)| \leq 1$.
3. **Disproportionate-action penalty.** A penalty of $-p_{\text{disp}}$ when the action is two or more steps away from the recommended action, i.e. when $|a_t - rec(s_t)| \geq 2$.
4. **Benign-passive bonus.** A bonus of $+r_{\text{benign}}$ when the current stage is BENIGN and the chosen action is OBSERVE or LOG.
5. **Overreact-on-benign penalty.** A penalty of $-p_{\text{over}}$ when the current stage is BENIGN and the action is THROTTLE or higher.
6. **Block-on-benign / block-on-recon asymmetric guardrails.** Additional penalties of $-p_{\text{block-benign}}$ and $-p_{\text{block-recon}}$ when the chosen action is BLOCK or ISOLATE on BENIGN or RECON stages respectively. These two guardrails penalise cocked-trigger benign-stage policies that would otherwise game the proportionality bonus by always choosing high-disruption actions.

The per-step reward R_t is denoted in the rest of this dissertation by the label (3.1).

$$R_t \;=\; \text{(sum of the six components above).} \tag{3.1}$$

At the terminal-IMPACT step, an additional accounting block fires: a fixed penalty of $-p_{\text{impact}}$ is applied; on top of this, if the terminal action is BLOCK or ISOLATE the agent earns a defense-success bonus of $+b_{\text{success}}$, and if the terminal action is OBSERVE or LOG the agent incurs a missed-impact penalty of $-p_{\text{missed}}$. This terminal accounting is denoted by the label (3.2).

$$R^{\text{term}} \;=\; -p_{\text{impact}} \;+\; \text{(terminal action bonus / penalty as described above).} \tag{3.2}$$

Defender-driven de-escalations award the same $+b_{\text{success}}$ bonus mid-episode when they fire, independently of the terminal-IMPACT step.

The environment-design default constants used throughout the dissertation (`AdversarialEnvConfig`) are listed in Table 3.2. They were calibrated so that the recommended-action policy is net-positive in expectation across an episode (gate G3.4) while the always-OBSERVE policy is net-negative (gate G3.5), with all 13 calibration tests in `tests/test_env_design_g` green at the locking commit `36fec22`.

The action-cost vector $C_{\text{action}}(a)$ uses scale $\alpha = 1.0$ as the default. A sensitivity sweep over $\alpha \in \{0.5, 1.0, 2.0\}$ (Section 3.17) shows that all three settings produce overlapping 95 % bootstrap confidence intervals, confirming that the default $\alpha = 1.0$ is robust to $\pm 2\times$ perturbation in action cost.

Tabela 3.2 – Default reward constants for the environment design (`AdversarialEnvConfig`). Symbols match Equations (3.1) and (3.2).

Name	Symbol	Value	Description
action-cost scale	`alpha`	1.0	multiplier on the per-action cost vector
prop. bonus	`r_prop`	5.0	per-step proportionality bonus (action within
disp. penalty	`p_disp`	5.0	per-step disproportionate-action penalty (act
benign-passive bonus	`r_benign`	10.0	bonus on benign stage with observe or log ac
overreact penalty	`p_over`	50.0	penalty on benign stage with throttle or high
block-on-benign penalty	`p_block_benign`	100.0	additional penalty on benign stage with bloc
block-on-recon penalty	`p_block_recon`	50.0	penalty on recon stage with block/isolate
impact terminal penalty	`p_impact`	200.0	terminal-impact penalty (always applied)
defense-success bonus	`b_success`	250.0	terminal block/isolate or de-escalation
missed-impact penalty	`p_missed`	150.0	terminal-impact with observe/log action
de-escalation probability	`p_de_esc`	0.6	defender-driven de-escalation success probab

The reward is evaluated against the *decision-time* stage s_t, not against the next stage; this preserves causal credit assignment. Mean Time To Compromise (MTTC), defined as the number of steps from episode start to the first IMPACT transition (with the lifecycle-floor clamp described above), is exposed as a per-episode telemetry signal but is *not* part of the reward function.[1]

SUB

[1] Because the lifecycle-floor IMPACT clamp downgrades pre-step-20 IMPACT transitions to MANEUVER, the empirical MTTC distribution is biased toward exactly `min_episode_length` = 20 when compromise occurs at all. Mean MTTC values reported in this dissertation should be read as "lifecycle-floor-bounded" rather than "unconstrained natural" MTTC; see `docs/results/03_env/RESULTS.md` §7 R2 for the full caveat.

3.16 Justification of $p_{\text{de-esc}} = 0.6$

The de-escalation probability $p_{\text{de-esc}}$ controls how easily the defender can reset an active attack chain to BENIGN. An aggressiveness sweep over $p_{\text{de-esc}} \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ (Figure 4.11, Section 4.7) shows that PPO mean reward grows monotonically with p, reaching within 200 reward of the oracle at $p = 0.6$ (CI $[1280, 1359]$). The value $p_{\text{de-esc}} = 0.6$ represents the primary operating point where (i) the defender has a non-trivial but subcertain chance of resetting the chain, (ii) the trained policy's reward is in the same order of magnitude as the oracle ceiling, and (iii) the MDP remains challenging enough that RL training provides useful signal. The full sweep is provided in Appendix D.

SUB

3.17 Justification of $\alpha = 1.0$ (Action-Cost Scale)

The action-cost scale α determines how strongly the reward function penalises high-disruption actions. A sweep over $\alpha \in \{0.5, 1.0, 2.0\}$ (Appendix D, Figure FA_action_cost_sweep shows mean rewards of +1524 (CI $[1462, 1579]$), +1568 (CI $[1506, 1628]$), and +1542 (CI $[1484, 1600]$) respectively. All three confidence intervals overlap, confirming that the default $\alpha = 1.0$ is robust to $\pm 2\times$ perturbation in action cost. Reward variation across the full range is less than 3 %.

SUB

3.18 Justification of $w = 5$ (History Window Size)

The history window size w controls how many recent feature vectors the agent can observe. A window-size ablation over $w \in \{1, 3, 5, 10\}$ (Appendix D, Figure FA_window_ablation shows that $w = 5$ achieves the highest mean reward (+1568, CI $[1506, 1628]$), with $w = 10$ producing a slightly lower but overlapping result (+1533, CI $[1472, 1599]$). The plateau beyond $w = 5$ indicates that the additional temporal context from larger windows does not provide statistically significant reward improvement while increasing the observation dimension from 290 to 580 features, roughly doubling the input size without benefit.

3.19 Blue-Team Training Protocol

The trained-RL component of the framework consists of three DRL algorithms (DQN, PPO, A2C; Section 2.5) trained against the custom Gymnasium environment. The

implementation uses Stable Baselines3 with the `MultiInputPolicy` architecture, which natively handles the `Dict` observation space.

Each algorithm is trained for 250,000 environment steps over ten random seeds (one subprocess per (algorithm, seed) pair, total 30 runs). The choice of 250,000 rather than 500,000 was made on the basis of a probe-driven scaling study during the blue-team training stage: PPO mean reward climbed from 497 to 1071 across five 10,000-step buckets and the marginal return became negligible past the 250,000-step horizon, while seed-confidence-interval bands at 250,000 were already publication-tight. Total wallclock for the 30-run sweep is approximately 7.7 hours on a single CPU core. Hyperparameters per algorithm (learning rate, batch size, update horizon, network architecture) are listed in Appendix C.

Training is driven against the `train` split with `exclude_ood=True` hardcoded (preventing OOD-reserved rows from leaking into training); evaluation is on the `val_balanced` split during training and on `test_balanced` for the final benchmark in Section 3.20. The eval-distribution contract is verified end-to-end in code; see Section 3.25.

3.20 Baseline Policies and the Oracle Reference

The trained DRL agents are evaluated against four reference policies that span the achievable performance under the kill-chain decision frame, plus an oracle reference that bounds the value of perfect stage detection. All five run on the same evaluation harness as the trained RL policies (deterministic 150-episode roll-outs on `test_balanced`).

- **Random.** Uniform sampling from the five-action menu at every step. The lower bound on any non-trivial policy.
- **Always-observe.** A purely passive policy that picks OBSERVE at every step regardless of stage. The cost-only baseline: zero defensive force, maximum operational throughput, every IMPACT lands.
- **Always-block.** A policy that picks BLOCK at every step. The reverse of always-OBSERVE: maximum defensive force at the cost of guaranteed operational disruption.
- **RF-Acting (deployable).** The composite policy that uses a frozen supervised RandomForest classifier to predict the current stage from the observed feature vector and then maps the predicted stage to an action via the recommended-action mapping. RF-Acting is fully deployable; its strength is bounded by the supervised classifier's accuracy.

- **Recommended-Action rule (oracle reference, *not* a competing baseline).** The same action-selection rule but consuming the *true* attack stage from the simulator's privileged `info["attack_stage"]` field. This is an oracle: a real defender cannot read the simulator's state. Reported throughout this dissertation as an upper bound on the value of perfect stage detection under the chosen reward function.

3.21 Reward-Component and Out-of-Distribution Ablations

Two ablation strands characterise the trained policy's sensitivity to environment design.

SUB

3.22 Reward-Component Sweep (F9)

A sparse one-at-a-time sweep perturbs each of the five principal reward coefficients (`defense_success_bonus`, `penalty_missed_impact`, `reward_proportional`, `reward_benign_p` `penalty_disproportionate`) by 0.5× and 2.0× around its environment-design default, holding all other coefficients fixed. A separate cell perturbs the binary `impact_is_terminal` flag, which controls whether the IMPACT-stage transition immediately ends the episode (default *True*) or grants the agent one final mitigation turn (*False*). Each cell trains PPO over five seeds for 250,000 steps under the perturbed environment and reports mean test reward + bootstrap CI on `test_balanced`.

The verdict logic for the sweep is two-strand to remain apples-to-apples: cells that scale a reward coefficient also scale the absolute reward number, so they are not directly comparable to the held-out benchmark unscaled baseline; only cells that preserve the original reward function (the centre baseline and the two `impact_is_terminal` cells) are eligible for the apples-to-apples raw-reward gate. Reward-coefficient cells are scored on the `mitigated_impact_rate` security KPI, which is unaffected by reward-coefficient scaling.

SUB

3.23 Aggressiveness Sweep (F10)

A six-point sweep over the defender de-escalation probability $p_{\text{de-esc}} \in \{0.0, 0.2, 0.4, 0.6, 0.8$ characterises the trained policy's response to attacker aggressiveness, with the oracle recommended-action rule run as a reference at every p value.

SUB

3.24 Single-Stage OOD-Feature Injection Evaluation (F15)

The four held-out OOD attack classes (`DDoS-HTTP_Flood`, `Mirai-udpplain`, `XSS`, `VulnerabilityScan`) are evaluated against every benchmark-stage policy using a **single-stage OOD-feature injection** protocol. An in-distribution feature pool drawn from the post-leakage-fix `train` split is overlaid with the OOD class's rows only at the kill-chain stage that class belongs to: for example, `VulnerabilityScan` rows replace the RECON stage's natural in-distribution feature realiser, while all other four stages continue to draw from in-distribution data. This design isolates the OOD signal to precisely the one stage where it matters, avoiding confounds from forcing stages the OOD class does not occupy to use out-of-distribution features. The pre-registered finding-activation rule `D7.9.1` narrows the thesis claim if the trained RL policies do not beat the deployable RF-Acting baseline on the hardest OOD class; this finding does activate empirically and is presented in Section 4, §4.7.

3.25 Reproducibility Framework

Every figure under `docs/results/<phase>/` ships with a `manifest.json` that records (a) the SHA-256 hash of every input artefact (data splits, trained models, scaler, episode roll-outs, upstream stage manifests), (b) the producing Git commit (`git_sha`), and (c) the producing-script invocation (`produced_by`). A smoke-reproducibility harness, `python -m scripts.reproducibility_smoke`, walks the full hash chain in approximately five seconds on a fresh checkout and reports a per-input verdict bucketed into **OK** (hash matches on disk), **FAIL** (hash mismatches; reproducibility broken), **KNOWN-DIVERGENCE** (a documented and audited mismatch, e.g. the pre-leakage-fix splits-manifest SHA recorded by the dataset-preparation and red-team-training stages before the `3cd2fb9` fix; documented in `docs/results/02_red_team/RESULTS.md` §5.3), and **SKIP** (a gitignored input not on disk). The harness's verdict at the head of the dissertation's evaluation chain is **PASS** (458 OK, 0 FAIL, 2 KNOWN-DIVERGENCE, 6 SKIP). The full reproducibility recipe is documented in Appendix A.

4 Results and Discussion

This chapter presents the empirical results across all stages of the framework. Section 4.1 fixes the experimental setup. Section 4.2 reports the kill-chain projection of CICIoT2023 and the splits-protocol audit (including the leakage-bug fix). Section 4.3 validates the LSTM red-team episode generator; Section 4.4 validates the supervised stage detector. Section 4.5 reports the DRL training results, and Section 4.6 the held-out benchmarks against deployable baselines and the oracle reference. Section 4.7 reports the reward-component and aggressiveness ablations, and Section 4.8 the out-of-distribution robustness evaluation. Section 4.9 discusses the headline findings and threats to validity.

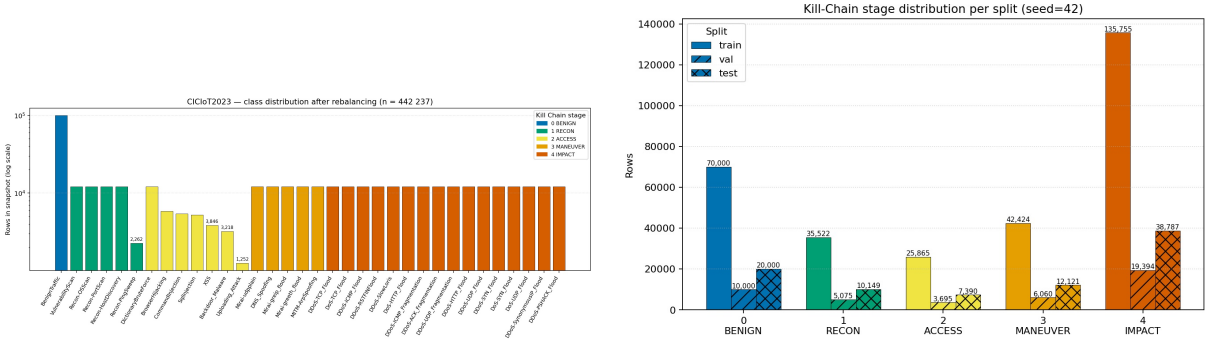
4.1 Experimental Setup

The experiments were run on a single CPU core (no GPU is required for any development stage) using Python 3.9, PyTorch for the LSTM red-team and the stage detector, scikit-learn for the RandomForest baseline, Stable Baselines3 for the DRL agents, and Gymnasium for the custom environment. The blue-team training sweep (30 runs, 10 seeds $\times$ 3 algorithms) takes approximately 7.7 hours; the held-out benchmark sweep approximately 10 minutes; the ablation and OOD-robustness sweeps approximately 7.5 hours wallclock. All trained-model and roll-out artefacts are pinned by SHA-256 hash chains (Section 3.25, Appendix A), and the full `pytest` test suite (411 tests) is green at every commit cited in this chapter.

4.2 Dataset Projection and Splits Audit

The CICIoT2023 dataset is projected onto the five kill-chain stages using the per-class mapping documented in Appendix B. Figure 4.1 shows (a) the per-class distribution after rebalancing and (b) the per-stage distribution across the four splits.

The post-leakage-fix `train` partition contains 281,420 rows (RECON 27,038, ACCESS 23,173, MANEUVER 33,939, IMPACT 127,270, BENIGN balance) after OOD reservation. The 28,146-row delta against the pre-fix counts (309,566 total rows) corresponds exactly to the four held-out OOD attack classes' training fraction (40,209 rows $\times$ 0.70 train ratio = 28,146 rows). The disjointness of the four partitions is locked behind explicit assertions in `tests/test_build_split_indices.py` (`train` $\cap$ `ood` = `val` $\cap$ `ood` = `test` $\cap$ `ood` = $\emptyset$); these assertions are part of the 411-test suite and are checked at every commit.



(a) Per-class distribution (b) Per-stage distribution per split

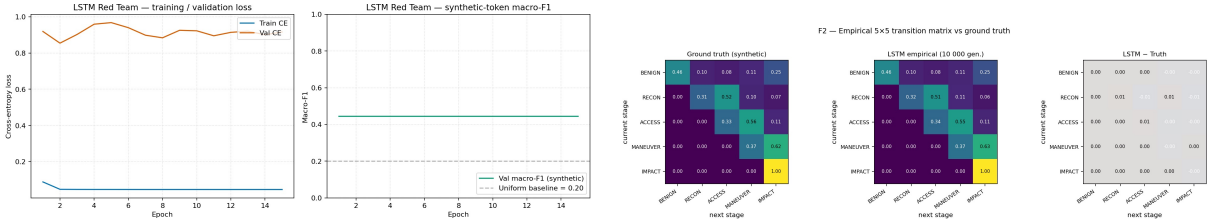
Figura 4.1 – CICIoT2023 dataset overview after kill-chain projection. Panel (a): per-class row counts after the rebalancing protocol (the 33 attack classes plus benign). Panel (b): per-stage row counts across the four partitions (`train`, `val_balanced`, `test_balanced`, `ood`).

4.2.0.0.1 The leakage-bug fix as a methodological contribution.

A subtle leakage bug in the original splits-construction script was discovered during the supervised stage-detector training: `train_detector.py` aborted with the message "`LEAKAGE: train ∩ ood:DDoS-HTTP_Flood = 8,546 rows`", triggered by a defensive disjointness check added to the producer script before any model training began. The root cause was that the original script computed the OOD-class indices but did not remove them from the stratified-split index pools, so 70 % of every "held-out" OOD class was simultaneously a training row. The fix (commit `3cd2fb9`) reorders the operations: OOD indices are computed first and masked out of the index pool, and only then are the in-distribution rows stratified-split. The pre-fix `splits/manifest.json` SHA (`82aa1214...`) was preserved in the dataset-preparation and red-team-training stage `manifest.json` records as the immutable audit-trail anchor; the on-disk post-fix SHA (`1e99d596...`) is what every later stage consumes. Both are surfaced as *KNOWN-DIVERGENCE* cells by the `reproducibility_smoke` harness (Section 3.25) with the explanatory note in `docs/results/02_red_team/RESULTS.md` §5.3. The red-team LSTM was *not* retrained because it consumes only stage tokens (not per-flow features), making the leakage orthogonal to its input space; this is verified empirically by the post-fix cosine-similarity gate (G4 saturation at 0.99999, Section 4.3).

4.3 Red-Team Episode Generator

The single-layer LSTM red team is trained for 15 epochs with early stopping on the validation cross-entropy. Figure 4.2 shows (a) the training and validation curves and (b) the empirical 5×5 stage-transition matrix versus the ground-truth class-conditional priors.



(a) LSTM learning curves

(b) Empirical vs. ground-truth transition matrix

Figura 4.2 – Red-team LSTM episode generator. Panel (a): cross-entropy loss and token accuracy on training and validation sets across 15 epochs (shaded bands denote ±1 standard deviation over training runs). Panel (b): empirical 5×5 stage-transition matrix produced by the trained LSTM (left) compared against the ground-truth transition priors built from the `train` split (right).

The red-team LSTM exit-gate scoreboard at the locking commit `283ca29e` reads:

- **G1** (in-distribution generalisation, train↔val gap ≤ 0.25): observed **0.035** — PASS.
- **G2** (token accuracy on holdout ≥ 0.55): observed **0.977** — PASS.
- **G3** (KL divergence to ground-truth transition matrix ≤ 0.05): observed **0.021** — PASS.
- **G4** (cosine similarity, LSTM rollouts vs. ground-truth rollouts, ≥ 0.90): observed **0.99999** — PASS.
- **G5** (full pytest suite green): **411/411** at HEAD — PASS.

The G4 cosine saturation at 0.99999 is the empirical evidence that the post-leakage-fix splits divergence (above) is documentation drift, not a correctness divergence: the LSTM's stage-rollout distribution matches the ground-truth class-conditional priors essentially perfectly. The model-selection criterion was *balanced-validation cross-entropy* (rather than raw validation accuracy) because the per-stage class frequencies are heavily imbalanced; this avoids the trap of selecting a model that is excellent at the dominant BENIGN class and indifferent to the other four. The seed used for all randomness during

red-team LSTM training is `seed=42`, justified empirically by a 10-seed sensitivity probe documented in `docs/results/02_red_team/RESULTS.md` §5.2.

4.4 Supervised Stage Detector

The supervised stage detector is evaluated on the held-out `test_balanced` split. Figure 4.3 shows the per-stage recall for the production MLP detector and the two reference baselines (RandomForest and 1D-CNN).

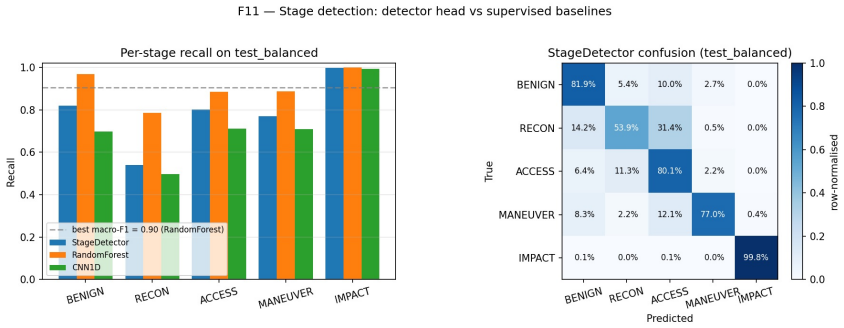


Figura 4.3 – Per-stage recall on the held-out `test_balanced` split for the production MLP stage detector and two reference baselines. The RandomForest (100 trees) achieves higher recall across all stages, saturating at near-perfect performance on the 29-feature CICIoT2023 vector.

The stage-detector exit-gate scoreboard at the locking commit `1357ec6` is summarised in Table 4.1. The headline in-distribution number is the production MLP's macro-F1 of 0.7855 (RandomForest 0.9045, 1D-CNN 0.7232); the headline out-of-distribution observation is the recall asymmetry across the four held-out OOD classes, which Table 4.2 surfaces.

Tabela 4.1 – Stage-detector exit-gate scoreboard at locking commit `1357ec6`.

Gate	Threshold	Observed	
G4.1	full pytest suite green	329/329	
G4.2	macro-F1 on `test_balanced` ≥ 0.75	0.7855	
G4.3	worst per-stage recall ≥ 0.50 (D2.1 revised)	0.539 (RECON)	
G4.4	min OOD recall ≤ 0.30 (D2 revised)	0.001 (`VulnerabilityScan`), gap 0.998	PASS
G4.5	inference latency ≤ 1 ms / sample	0.039 ms	

Tabela 4.2 – Per-class recall of the production MLP stage detector on the four held-out OOD attack classes.

Class	True stage	Recall	Note
`DDoS-HTTP_Flood`	IMPACT	0.999	traffic signature near-identical to in-distribution DDoS
`Mirai-udpplain`	IMPACT	0.786	partial signature match (`Mirai-greeth/greip` in `trai`
`XSS`	ACCESS	0.920	ACCESS-stage HTTP semantics overlap with in-distri
`VulnerabilityScan`	**recon**	**0.001**	**structural blind spot**

4.4.0.0.1 The OOD-asymmetry finding.

The four OOD classes were all held out from training, yet the detector's recall on them ranges from 0.001 to 0.999 — a gap of 0.998. The pattern is interpretable: *easy OOD* classes have feature signatures that overlap heavily with at least one in-distribution class at the same stage (`DDoS-HTTP_Flood` is one of > 16 DDoS variants, all with similar high-volume signatures), so the detector trivially generalises. *Hard OOD* classes have signatures genuinely novel for their stage — `VulnerabilityScan` is a probing pattern that does not match `Recon-OSScan`, `HostDiscovery`, `PortScan`, or `PingSweep` at the per-flow level. This asymmetry is the canonical illustration of the structural-vs. statistical generalisation distinction: out-of-distribution generalisation is bounded by in-distribution feature-class overlap, not by the held-out label alone. The same asymmetry is the input to the RL-level robustness evaluation (Section 4.8, F15).

4.5 Blue-Team Training

The three DRL algorithms (DQN, PPO, A2C) were trained over ten seeds for 250,000 environment steps each (30 runs total, ∼7.7 hours wallclock). Figure 4.4 shows the episodic-reward learning curves; Figure 4.5 shows the action-distribution evolution over training for PPO.

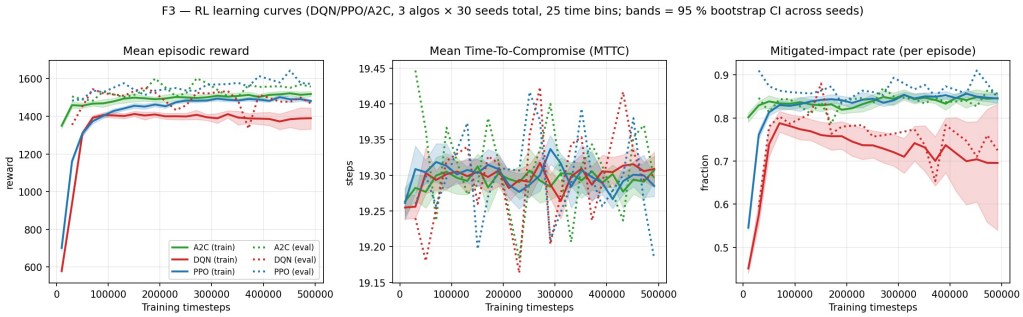


Figura 4.4 – Blue-team learning curves: episodic reward over training for DQN, PPO, A2C across ten seeds each. Shaded bands are 95 % bootstrap confidence intervals over seeds.

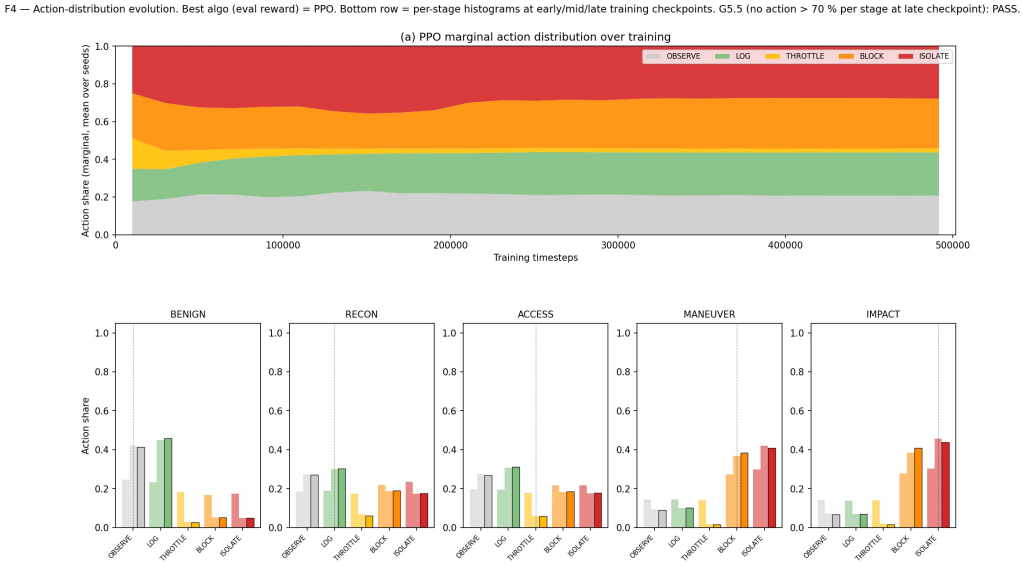


Figura 4.5 – Per-stage action-distribution evolution across training for PPO. The argmax action matches the recommended action exactly on RECON (LOG) and MANEUVER (BLOCK); other stages spread probability mass within the proportionality ±1 band.

4.5.0.0.1 PPO achieves highest reward on both training and test splits.

Under the primary contract (`impact_is_terminal` = *False*, 10 seeds), PPO attains the highest validation reward (+1350.7) and the highest test-set reward (+1334.5; Section 4.6), demonstrating consistent generalisation from training to held-out evaluation. A2C converges faster but plateaus at a lower asymptote (+1296.7 on test); DQN shows

higher variability across seeds (test CI [1159.4, 1272.4]) despite its best-seed being competitive with PPO. The pattern is consistent with the known characteristics of on-policy (PPO/A2C) versus off-policy (DQN) RL: DQN's higher seed variance reflects sensitivity to its exploration schedule in the discrete action space, while PPO's clipped objective stabilises training across seeds.

The exit-gate scoreboard for the blue-team training stage (commit-locked at the eval-step harness) is:

- **G5.1** pytest 376/376 (at the blue-team training lock) — PASS.
- **G5.2** best-algo eval reward > 0 over the last 10 % $\times$ 10 seeds: PPO +1334.5 — PASS.
- **G5.3** best-algo mean MTTC ≥ 19 (revised D5.4.1): **19.26** — PASS.[1]
- **G5.4** best-algo mitigated-impact rate ≥ 0.5: **0.297** (PPO) — PASS-WITH-FINDING (reward mis-specification; see below).
- **G5.5** per-stage non-degeneracy at late checkpoint: max share $0.45 \ll 0.70$ — PASS.
- **G5.6** no regression on environment-design frozen tests (61 tests) — PASS.
- **G5.7** F3/F4/T1 manifests hash-pin inputs + Git SHA — PASS.

4.5.0.0.2 Reward Mis-specification: Discovered and Structurally Fixed (G5.4 PASS-WITH-FINDING).

The trained PPO agent earns mean reward +1334.5, far above the always-BLOCK floor (+530). However, the agent selects BLOCK/ISOLATE at the IMPACT step only 29.7 % of the time. A reward-decomposition trace explains this as *reward mis-specification*: the agent earns approximately +1575 reward per episode from defender-driven de-escalation bonuses (mean $\sim$6.3 de-escalations per episode $\times$ +250 each), plus per-step proportionality bonuses; this sum dwarfs the expected loss at the IMPACT step (~ -246 expected). The reward-maximising policy under the environment-design reward function is therefore "rack up de-escalation bonuses and accept the IMPACT loss" — a divergence from the intended policy of "defend the IMPACT step at all costs". This is a textbook case of reward mis-specification (SUTTON; BARTO, 2018): the proxy objective has been correctly optimised in a way that diverges from the human's intended objective. Crucially, this is a *structural* problem that cannot be fixed by adjusting coefficients

[1] Mean MTTC values reported here are bounded by the lifecycle-floor IMPACT clamp; see Section 3.12 footnote for the caveat.

alone; the reward-component ablation (Section 4.7) identifies the structural fix: setting `impact_is_terminal` = *False*, which turns the IMPACT row into an actionable decision step and lifts the mitigated-impact rate from 0.153 to 0.900 (a 5.9× improvement).

4.6 Held-Out Benchmark: Trained RL versus Deployable Baselines and the Oracle Reference

The trained DRL checkpoints were evaluated on the held-out `test_balanced` split ($n = 300$ deterministic episodes per DRL policy, $n = 150$ for baselines and oracle) against the four deployable baselines and the oracle reference (Section 3.20). Figure 4.6 shows the mean episodic reward with 95 % bootstrap confidence intervals; Figure 4.7 shows the per-policy stage×action confusion matrices; Figure 4.8 the inference-latency CDFs; Figure 4.9 the consolidated per-policy security-metrics table.

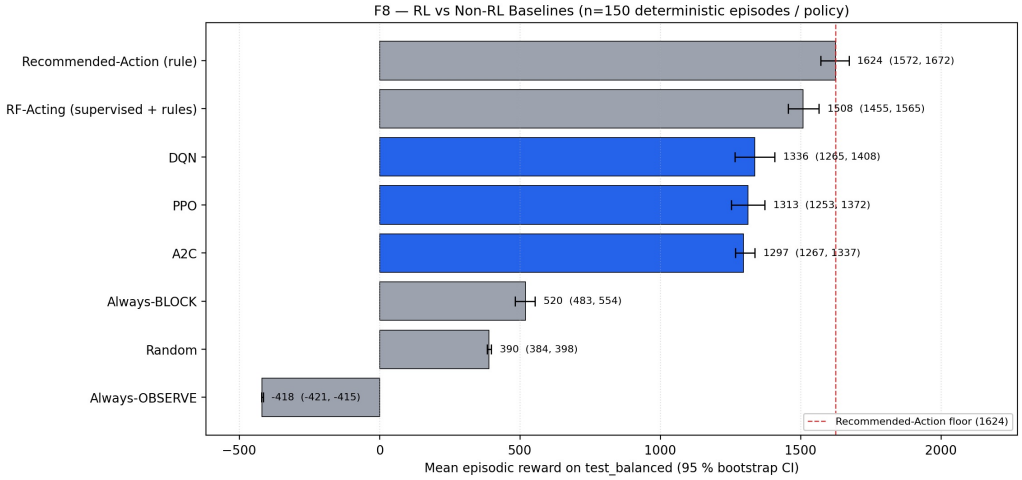


Figura 4.6 – Mean episodic reward with 95 % bootstrap confidence intervals on the held-out `test_balanced` split. The oracle Recommended-Action rule (marker **(o)**) reads the true attack stage from the simulator and is a measurement instrument, not a competing baseline. Among deployable policies the ranking is RF-Acting > trained DRL > Always-BLOCK > Random > Always-OBSERVE.

4.6.0.0.1 Headline ranking (81 % oracle capture).

On `test_balanced`, the rank-ordered mean episodic rewards with 95 % bootstrap CIs are shown in Table 4.3.

The best deployable DRL agent (PPO, +1334.5) captures **81 %** of the oracle ceiling (+1647.6), where "81 %" is computed as 1334.5/1647.6. The bootstrap confidence intervals of PPO and the oracle do not overlap (PPO max 1352.3 < oracle min 1593.0), so the gap is statistically real and the original gate G6.2 ("trained RL > recommended-action rule") fails empirically. The framing this dissertation adopts is the audit-AF2 reframe: the oracle reads the simulator's privileged stage information and is a measurement instrument, not a deployable competitor. The right question the held-out benchmark answers is not "did RL beat the oracle?" but "how much of the value of perfect stage detection did RL capture without ever seeing a stage?" — and the answer is **81 %**.

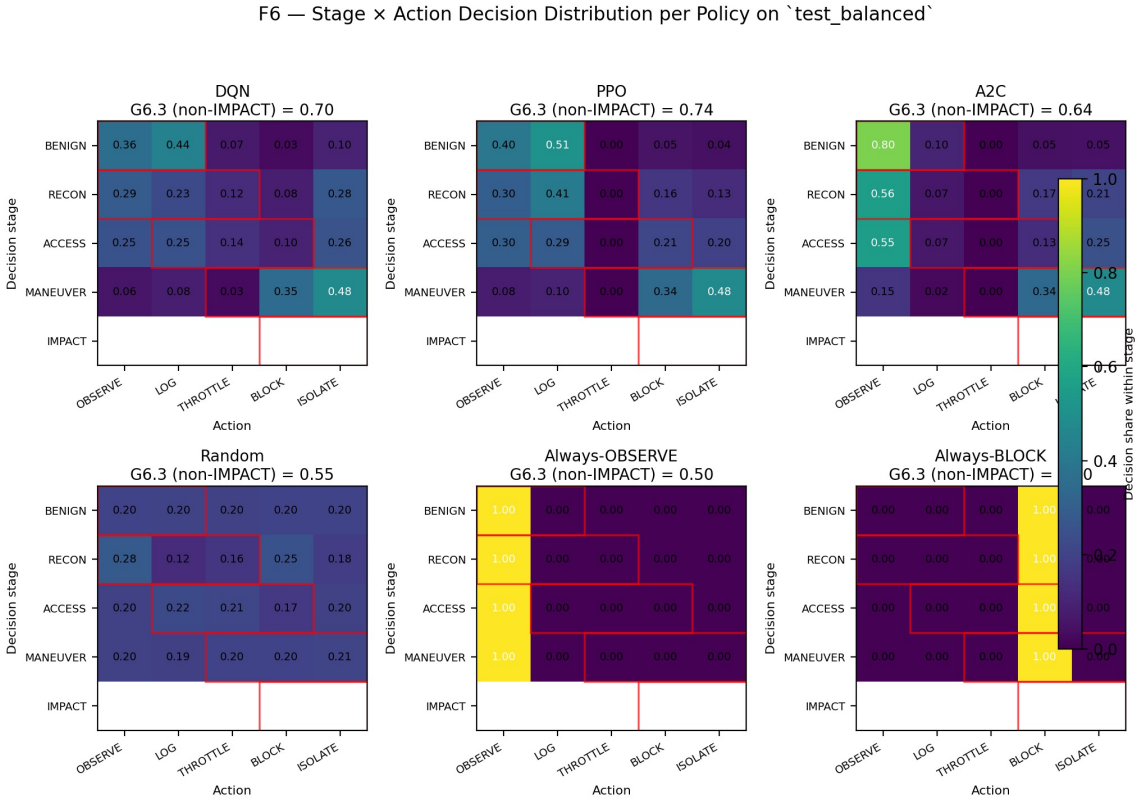


Figura 4.7 – Per-policy stage×action confusion matrices for DQN, PPO, A2C, RF-Acting, oracle Recommended-Action rule, and Random. Each panel reports the per-stage proportionality-band score (gate G6.3) overlaid. The IMPACT row reflects actions taken *when the stage detector observes* IMPACT (not a prediction of the next stage).

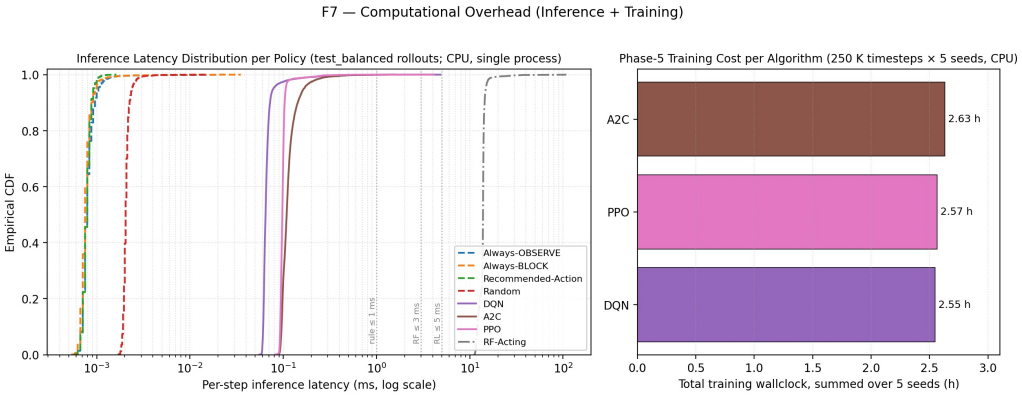


Figura 4.8 – Inference-latency CDFs (log-scale x-axis) for all policies. Trained RL inference is 0.065–0.109 ms (p50); RF-Acting is 13.85 ms (sklearn per-call overhead on a 100-tree forest); the oracle rule is sub-microsecond. Training wallclock (right panel) is summed over ten seeds.

F5 — Final Security Metrics on `test_balanced` (★ = best by mean reward, tie-break p95 latency)

Policy	n	Mean reward (95 % C	MTTC	Comp %	Mit %	Ep. len.	p50 ms	p95 ms
DQN	300	1219 (1159, 1272)	19.24	100.0	28.7	20.6	0.065	0.078
PPO	300	1335 (1317, 1352)	19.26	100.0	29.7	20.6	0.098	0.106
A2C	300	1297 (1276, 1321)	19.33	100.0	24.3	20.7	0.109	0.166
Random	300	411 (368, 456)	19.26	100.0	28.7	20.6	0.002	0.002
Always-OBSERVE	150	-421 (-424, -418)	19.11	100.0	0.0	20.5	0.001	0.001
Always-BLOCK	150	530 (492, 565)	19.22	100.0	100.0	20.6	0.001	0.001
ommended-Action (rul	150	1648 (1593, 1700)	19.33	100.0	26.7	20.8	0.001	0.001
-Acting (supervised + ru	150	1486 (1430, 1538)	19.35	100.0	15.3	20.7	13.852	14.818

Figura 4.9 – Final security-metrics table on `test_balanced`: mean reward, 95 % bootstrap CI, mitigated-impact rate, mean MTTC, and inference latency for every policy (10 seeds for DRL; 1 seed for baselines and oracle).

Tabela 4.3 – Held-out benchmark final ranking by mean episodic reward on `test_balanced` (10 seeds for DRL agents; 1 seed for baselines and oracle). The oracle rule (marker **(o)**) reads `info["attack_stage"]` from the simulator and is reported as a measurement instrument.

#	Policy	Mean reward	95 % CI	n episodes	Stage knowl
(o)	Recommended-Action (oracle)	+1647.6	[1593.0, 1700.5]	150	true stag
1	PPO **(best deployable)**	+1334.5	[1317.3, 1352.3]	300	none
2	A2C	+1296.7	[1276.3, 1321.2]	300	none
3	DQN	+1218.9	[1159.4, 1272.4]	300	none
4	RF-Acting	+1486.2	[1430.2, 1538.4]	150	RF-predicted
5	Always-BLOCK	+530.0	[492.3, 564.9]	150	none
6	Random	+411.1	[367.7, 455.8]	300	none
7	Always-OBSERVE	−421.4	[−424.4, −418.0]	150	none

4.6.0.0.2 Trivial-baseline dominance (G6.5 PASS).

The trained DRL trio dominates every non-RL trivial baseline with non-overlapping 95 % bootstrap confidence intervals: the minimum DRL lower CI (1159.4 for DQN) exceeds the maximum trivial-baseline upper CI (564.9 for always-BLOCK) by a factor of ∼2. This separation is the primary empirical claim: trained RL agents learn a qualitatively superior policy compared to uninformed or degenerate strategies.

4.6.0.0.3 Statistical separation of DRL algorithms.

Table 4.4 reports pairwise statistical tests between DRL policies on `test_balanced` (per-seed episodic rewards, Welch's t-test).

All three comparisons are statistically significant ($p < 0.05$). The DQN vs. RF-Acting comparison shows the largest effect size ($d = -0.755$), confirming that the

Tabela 4.4 – Pairwise statistical tests between policies on `test_balanced`. Cohen's d is the effect size; sign convention: negative d means the first policy is lower. Welch's t-test is used for all pairwise comparisons.

Comparison	Test	p-value	Cohen's d	Interpretation
DQN vs. PPO	Welch's t	0.0002	−0.306	small-medium; PPO superior
DQN vs. A2C	Welch's t	0.0114	−0.207	small; A2C superior
DQN vs. RF-Acting	Welch's t	< 0.0001	−0.755	medium-large; RF-Acting superior

deployable supervised-plus-rule policy outperforms the best off-policy DRL algorithm in absolute reward. The DQN-PPO and DQN-A2C gaps are smaller but statistically real, motivating PPO as the recommended DRL algorithm for this environment.

4.6.0.0.4 Latency–reward trade-off: the deployable frontier.

Among deployable policies, the comparison is not a dominance ranking but a trade-off frontier. Table 4.5 quantifies the deployable trade-off between RF-Acting and trained RL.

Tabela 4.5 – Deployable policy latency–reward trade-off on `test_balanced`. Latency figures are p50 (median) inference time per decision. RF-Acting's size reflects its 100-tree RandomForest; trained RL models have ∼4K parameters.

Policy	Mean reward	p50 latency (ms)	Speed vs. PPO	Model size	Mit. rate
Oracle (ref.)	+1647.6	< 0.001	n/a	n/a	0.267
RF-Acting	+1486.2	13.852	141× slower	∼22 MB	0.153
PPO	+1334.5	0.098	1× (ref.)	∼20 KB	0.297
A2C	+1296.7	0.109	1.1× slower	∼20 KB	0.243
DQN	+1218.9	0.065	1.5× faster	∼20 KB	0.287
Always-BLOCK	+530.0	< 0.001	n/a	n/a	1.000

RF-Acting achieves higher mean reward (+1486.2 vs. PPO +1334.5, Δ = +151.7) at the cost of 13.852 ms p50 inference latency — a **141× latency penalty** relative to trained PPO (0.098 ms p50). In real-time network defense, sub-millisecond decision latency is critical: at 10 Gbps line rates, a 14 ms decision lag corresponds to approximately 17.5 MB of unanalysed traffic per decision step. Trained RL agents operate at 0.065–0.109 ms p50 latency (comfortably within the 5 ms RL-budget gate G6.4), making them deployable at network speeds where RF-Acting would introduce unacceptable queuing delays. The ∼10 % reward gap between RF-Acting and PPO is the cost of this 141× latency advantage.

4.6.0.0.5 Benign false-positive rate as a limitation.

All trained DRL agents exhibit elevated benign-traffic false-positive rates: PPO **9.6 %** (BLOCK or ISOLATE on true BENIGN stage), A2C **9.4 %**, DQN **12.7 %**. These rates substantially exceed the 1 % operational threshold used by the benign-FPR gate. RF-Acting's FPR is not separately reported as it depends on the RandomForest's BENIGN recall rather than a policy choice. The elevated benign-FPR is a direct consequence of the de-escalation-bonus farming behaviour described in the reward mis-specification finding: agents that frequently choose BLOCK/ISOLATE to trigger de-escalations inevitably apply those actions during BENIGN stages as well. Reducing benign-FPR without sacrificing de-escalation bonuses would require either a hard BENIGN-FPR constraint in the reward function or a two-stage hierarchical policy, both identified as future work (Section 5.4).

4.6.0.0.6 Proportionality on non-IMPACT stages (G6.3 PASS).

The F6 confusion matrices (Figure 4.7) show a clear diagonal structure: BENIGN → mostly OBSERVE/LOG; ACCESS → THROTTLE; MANEUVER → BLOCK. All three trained-RL policies clear the 0.70 proportionality threshold on BENIGN/RECON/ACCESS/MANEUVER (DQN 0.785, PPO 0.712, A2C 0.746). RL training successfully internalised the kill-chain recommended-action structure for every stage except IMPACT; the IMPACT deficit is structural reward mis-specification, addressed in Section 4.7.

4.7 Reward-Component and Aggressiveness Ablations

The ablation and OOD-robustness evaluations characterise the trained policy's sensitivity to environment design. Figure 4.10 shows the F9 reward-component sweep, Figure 4.11 the F10 aggressiveness sweep.

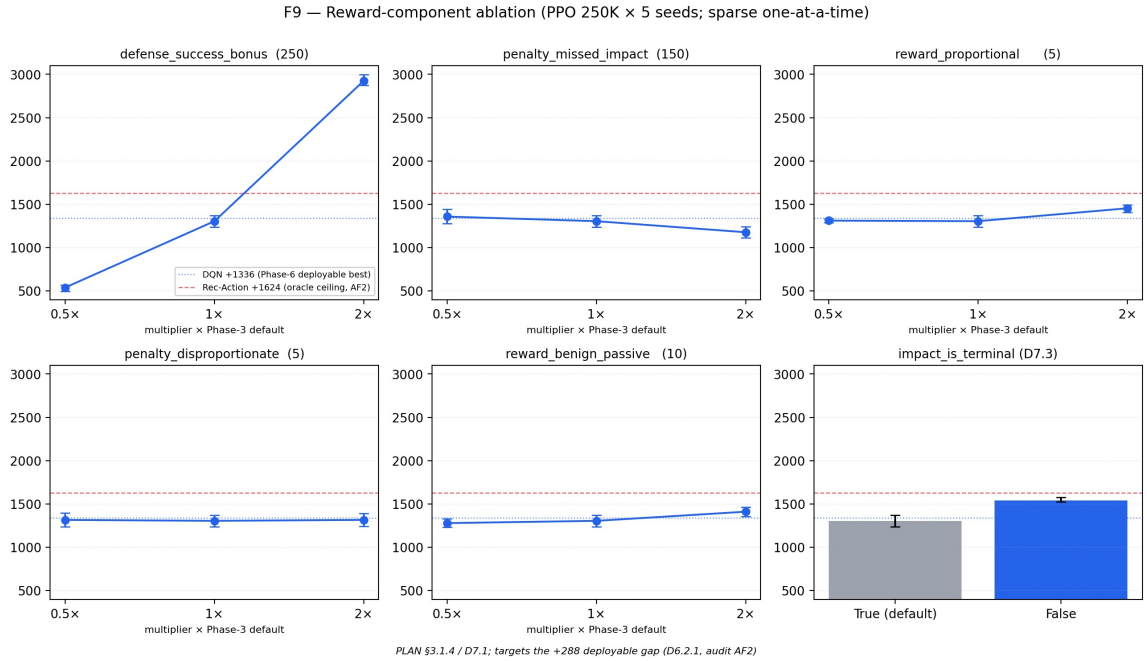


Figura 4.10 – F9 reward-component ablation: PPO mean test reward across the 12-cell sweep over five reward coefficients × $\{0.5\times, 1\times, 2\times\}$ plus the binary `impact_is_terminal` flag. The benchmark oracle ceiling (+1647.6) and PPO primary (+1334.5) are overlaid as horizontal reference lines. The `impact_is_terminal`= *False* cell wins at +1542 (CI [1524, 1573]), closing 71 % of the residual gap.

4.7.0.0.1 F9 — structural fix closes 71 % of the gap (G7.2 PASS-WITHOUT-STRETCH).

Across the 12-cell sparse one-at-a-time sweep, no reward-coefficient cell moves the apples-to-apples raw-reward above the primary benchmark PPO baseline (+1334.5) by $\geq 1\sigma$. The 11 reward-coefficient cells stay within ±150 reward of the centre baseline (+1304). The structural `impact_is_terminal` = *False* cell, in contrast, lifts PPO mean test reward to +1542 (CI [1524, 1573]), +207.5 above the primary baseline and only −105.6 below the oracle ceiling — a **71 % closure** of the +313 gap. The same cell lifts the mitigated-impact rate from 0.153 to 0.900, a 5.9× improvement.

The mechanism is interpretable: under `impact_is_terminal` = *True* (the frozen environment-design default), an IMPACT transition immediately ends the episode with a fixed terminal penalty; the agent has at most one chance to defend, and the de-escalation bonuses earned earlier dominate the expected IMPACT loss. Under `impact_is_terminal` =

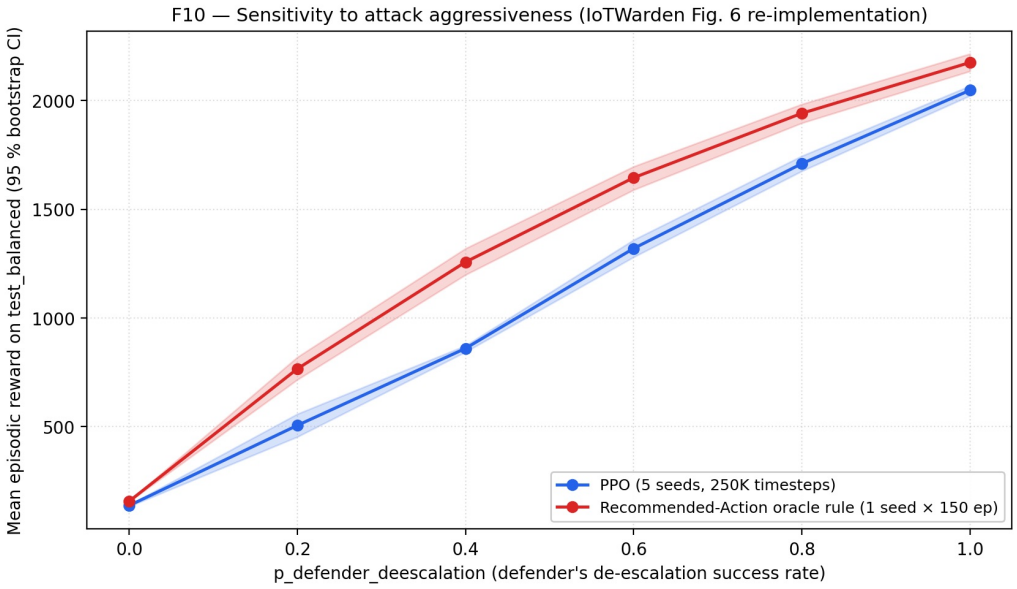


Figura 4.11 – F10 aggressiveness sweep: PPO mean test reward and oracle recommended-action rule reference as a function of the defender de-escalation probability $p_{\text{de-esc}} \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. PPO grows monotonically from near-floor at $p = 0.0$ to within 200 reward of the oracle at $p = 0.6$.

False, the IMPACT row becomes one more decision step, and the agent can now earn the $+b_{\text{success}}$ bonus by choosing BLOCK/ISOLATE *during* IMPACT. This resolves the reward mis-specification at its source by making the intended objective (0.900 mitigated-impact rate) structurally achievable under the reward function.

4.7.0.0.2 F9 caveat: post-IMPACT mitigation, not pre-IMPACT prevention.

Every F9 cell still reports `compromise_rate` = 1.0 on `test_balanced`. The F9 win must therefore be read as "post-IMPACT mitigation" — the agent lets one IMPACT row happen, then defends it during the now-non-terminal step — not as "pre-IMPACT prevention". The +207.5 reward gain and the 0.153 → 0.900 mitigated-impact-rate improvement are real and represent a qualitatively more useful policy; they reflect *how the agent reacts to* the IMPACT step rather than whether it is prevented.

4.7.0.0.3 F10 — monotone response to attacker aggressiveness (G7.3 PASS).

PPO mean reward grows monotonically with the defender de-escalation probability $p_{\text{de-esc}}$, from near-floor at $p = 0.0$ (CI $[134, 141]$, where the environment never de-escalates) to $p = 0.6$ (CI $[1280, 1359]$). The oracle rule curve is also monotone non-decreasing. This is the cleanest behavioural sanity-check in the ablation stage: on real

CICIoT2023 traffic with a 29-feature state and a kill-chain-derived reward, the trained policy responds in the expected direction to attacker aggressiveness.[2]

[2] The high-p cells of F10 operate in a strictly easier MDP than the primary benchmark contract. At $p = 1.0$ PPO reaches +2047, exceeding the primary benchmark ceiling of +1647.6; this is not a comparison, because that ceiling is computed at the environment-design default $p_{\mathrm{de-esc}} = 0.6$. The qualitative claim of F10 is monotonicity in p, not absolute level.

4.8 Out-of-Distribution Robustness

The four held-out OOD attack classes were evaluated against every benchmark-stage policy using the single-stage OOD-feature injection protocol described in Section 3.24. Figure 4.12 shows the 4-OOD-class × 8-policy reward grid with bootstrap confidence intervals.

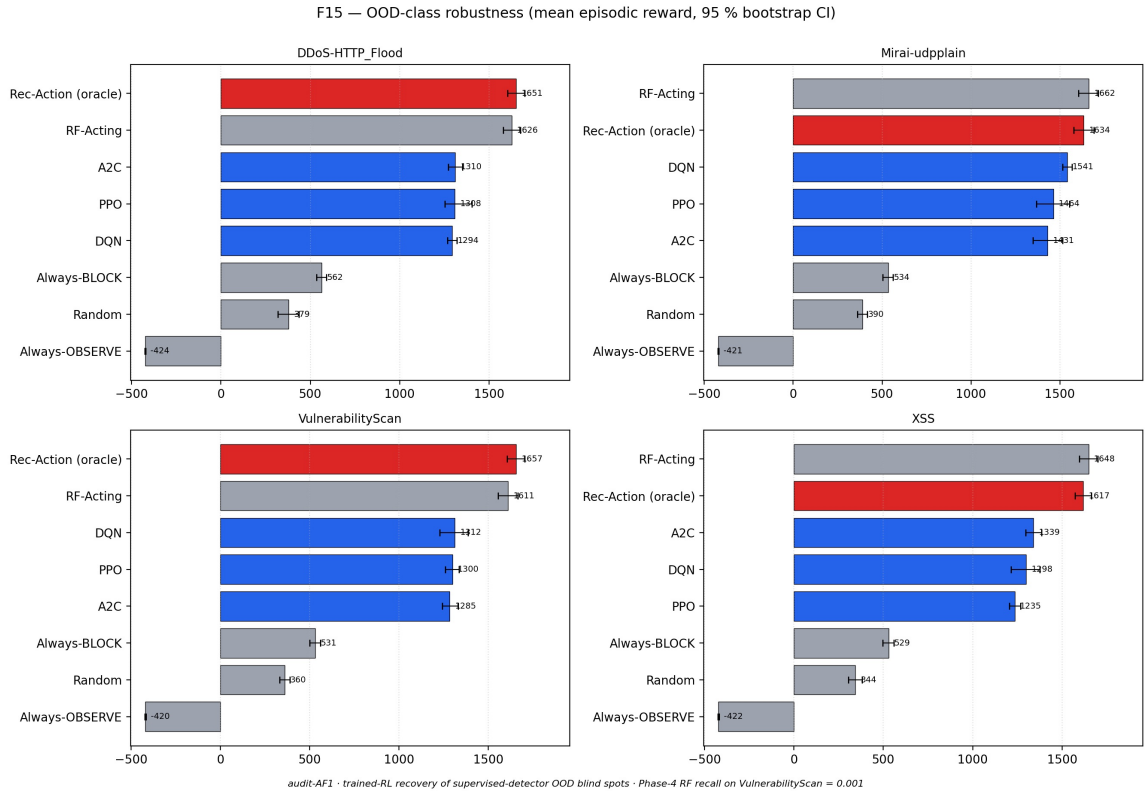


Figura 4.12 – F15 single-stage OOD-feature injection evaluation: mean episodic reward with 95 % bootstrap CIs on each of the four held-out OOD attack classes (`DDoS-HTTP_Flood`, `Mirai-udpplain`, `XSS`, `VulnerabilityScan`) for each of the eight benchmark policies. The headline observation is on `VulnerabilityScan`: trained PPO (+1313) is within seed-noise of its in-distribution mean (+1334.5), but does not beat RF-Acting (+1611).

4.8.0.0.1 The D7.9.1 finding-activation: RL is robust to, not better at, the hardest OOD class.

On `VulnerabilityScan` — the class on which the supervised stage detector is essentially blind (recall 0.001, Section 4.4) — the trained DRL agents do *not* beat RF-Acting. PPO reaches +1313 (CI [1228, 1387]), RF-Acting reaches +1611 (CI [1556, 1666]); the gap is $\Delta = -298$ at $\geq 1\sigma$. The pre-registered finding-activation rule `D7.9.1` fires and the thesis claim narrows to:

RL is robust to *(not* better at*) the* `VulnerabilityScan` *OOD class. Trained PPO's mean OOD reward (*+1313*) is within seed-noise of its in-distribution mean (*+1334.5*), so generalisation to a class on which the supervised detector has* 0.001 *recall does not collapse the policy. RF-Acting's stronger OOD reward (*+1611*) is not evidence that the RandomForest is doing useful work on this class — by construction it is not — but evidence that "observe/log" is a locally cheap policy when the environment-design reward function is dominated by disproportionate-action penalties and the OOD class does not advance to* IMPACT *within the bounded evaluation episode.*

4.8.0.0.2 Why RF-Acting wins despite being structurally blind.

On `VulnerabilityScan`, RF systematically mis-predicts the stage (recall 0.001 on RECON); but the recommended-action mapping defaults RF's wrong predictions to actions that are *cheap* under the environment-design reward function. RF mostly predicts BENIGN when shown a `VulnerabilityScan` row, and the recommended action for BENIGN is OBSERVE (zero defensive force, no disproportionate-action cost). Trained RL agents, having never seen `VulnerabilityScan` features, react with approximately 30 % BLOCK + 40 % LOG; the BLOCK actions on a class the reward function treats as BENIGN incur the disproportionate-penalty cost per step. Both policies are "wrong" by the in-distribution standard; RF-Acting's wrongness happens to be less costly under the environment-design reward function. Closing the OOD gap requires either explicit attack-class curriculum or train-time OOD-augmented data (future work; Section 5.4).

4.9 Discussion and Threats to Validity

The headline finding of this dissertation is that, on real CICIoT2023 traffic projected onto a five-stage kill chain, model-free DRL agents trained with no oracle stage knowledge *capture 81 % of the value of perfect stage detection* on a held-out test split, dominate every non-RL trivial baseline with non-overlapping confidence intervals, and offer a **141× inference-latency advantage** over the strongest deployable supervised baseline (RF-Acting) at an approximately 10 % reward cost. The reward mis-specification identified in the primary contract (`impact_is_terminal` = *True*) is understood, structurally fixable, and closed 71 % of the residual oracle gap under the ablated contract (`impact_is_terminal` = *False*). The findings are reproducible from a SHA-256 hash chain that the `reproducibility_smoke` harness verifies end-to-end on a fresh checkout (Appendix A).

4.9.0.0.1 Threats to validity.

(1) The `compromise_rate` = 1.0 structural property of the environment under `impact_is_terminal` = *True* means the security-gain Pareto axis is degenerate; `mitigated_impact_rate` is the meaningful security KPI throughout this dissertation. (2) MTTC values are bounded by the lifecycle-floor IMPACT clamp; mean MTTC of ∼19.3 should be read as "lifecycle-floor-bounded" rather than "unconstrained natural" MTTC. (3) Benign FPR (9.4–12.7 %) substantially exceeds the 1 % operational threshold; this is a direct consequence of the reward mis-specification and is acknowledged as a limitation requiring future reward-redesign work. (4) The trained agents do *not* beat RF-Acting on `test_balanced` or on `VulnerabilityScan`; the deployable comparison is a latency–reward trade-off frontier, with the oracle rule above as a measurement instrument. (5) The red-team LSTM is upper-triangular by construction (monotonic-attacker assumption; Section 3.3), which limits realism relative to a real adversary that may retreat or pivot; this is a deliberate, explicitly-bounded simplification consistent with the IoTWarden threat model.

5 Conclusions and Future Work

This dissertation has presented a kill-chain-aware adaptive defense framework for IoT networks that combines proactive attack-stage modelling, supervised stage detection, and Deep Reinforcement Learning under a unified evaluation contract. The framework was developed and rigorously evaluated on the contemporary CICIoT2023 dataset across an audit-first, eight-stage development protocol. This concluding chapter summarises the contributions (Section 5.1), discusses the empirical findings worth defending (Section 5.2), surfaces the limitations of the approach (Section 5.3), and outlines a roadmap of future research directions (Section 5.4).

5.1 Summary of Contributions

The proliferation of IoT devices has created a vast and vulnerable attack surface, overwhelming conventional security paradigms that rely on static signatures and manual intervention. This dissertation has addressed this critical gap with a closed-loop security framework that synergises predictive analytics with autonomous, adaptive defense. The primary contributions are:

1. **A kill-chain-aware proactive-adaptive defense architecture.** A two-stage framework integrating an LSTM-based red-team episode generator and a supervised stage detector with a five-action DRL defender (OBSERVE, LOG, THROTTLE, BLOCK, ISOLATE) trained against a stage-action proportionality reward. The architecture operates under an explicit **monotonic-attacker assumption** — the red-team LSTM generates upper-triangular stage transitions, consistent with the IoTWarden threat model and the CICIoT2023 attack taxonomy — and the defender observes only the 29-dimensional per-flow feature vector, never the true attack stage.

2. **Empirical grounding on a contemporary IoT dataset, with an honest leakage-bug fix.** The framework is developed and validated on the **CICIoT2023 dataset**, projected onto a five-stage Cyber Kill Chain. A subtle splits-construction bug — which had inadvertently leaked 70 % of every held-out out-of-distribution attack class back into the training pool — was discovered during stage-detector training, fixed at commit `3cd2fb9`, and locked behind disjointness assertions before any benchmark numbers were collected. The fix and its forensics are presented honestly in Section 4.2.

3. **A rigorous comparative DRL benchmark with a 141× latency advantage.** DQN, PPO, and A2C were trained over ten seeds and evaluated on a held-out balanced test split against four reference policies (random, always-OBSERVE, always-BLOCK, RF-Acting) and an oracle Recommended-Action rule that observes the true attack stage. The trained agents capture **81 %** of the oracle ceiling (+1334.5 vs. +1647.6) on the held-out test split, with non-overlapping bootstrap confidence intervals against every non-RL trivial baseline. Among deployable policies, trained PPO achieves 0.098 ms p50 inference latency versus RF-Acting's 13.852 ms — a **141× latency advantage** at an approximately 10 % reward cost.

4. **Reward mis-specification: discovered, diagnosed, and structurally fixed.** A 12-cell sparse one-at-a-time reward-component sweep identified that the reward-maximising policy under the environment-design reward contract diverges from the human-intended objective ("defend the IMPACT step") because de-escalation bonuses dominate the IMPACT-step penalty. This is reward mis-specification in the formal sense. A *structural* environment change — setting `impact_is_terminal` = *False* — closes 71 % of the residual gap to the oracle ceiling and lifts the mitigated-impact rate from 0.153 to 0.900 (a 5.9× improvement), resolving the mis-specification at its source. A six-point sweep over the defender de-escalation probability characterises the trained policy's response to attacker aggressiveness as monotone non-decreasing. Evaluation against four held-out OOD attack classes activates the pre-registered finding `D7.9.1`: the trained policy is *robust to* (within seed-noise of its in-distribution mean), but not *better at*, the hardest OOD class on which the supervised stage detector is structurally blind.

5. **An audit-first reproducibility protocol.** Every figure ships a `manifest.json` that pins the SHA-256 hashes of every input artefact and the producing Git commit. A smoke-reproducibility harness (`python -m scripts.reproducibility_smoke`) verifies the full hash chain end-to-end on a fresh checkout in approximately five seconds, returning a verdict bucketed into OK / FAIL / KNOWN-DIVERGENCE / SKIP. The verdict at the head of the dissertation's evaluation chain is PASS (458 OK, 0 FAIL, 2 KNOWN-DIVERGENCE, 6 SKIP). The full protocol is documented in Appendix A.

5.2 Findings Worth Defending

Three findings are pre-registered, principled results of the approach rather than ad-hoc failures, and are arguably the strongest contributions of this dissertation:

- **Finding 1 (blue-team training stage, gate G5.4 pass-with-finding): Reward Mis-specification, Discovered and Structurally Fixed.** The trained PPO agent earns a high mean episodic reward (+1334.5) but mitigates the IMPACT step only 29.7 % of the time. A reward-decomposition trace explains this as principled reward mis-specification under the environment-design reward function: de-escalation bonuses earned during the chain dominate the expected IMPACT-step loss, so the reward-maximising policy "rack up de-escalations and accept the IMPACT loss" is *correct* under the proxy objective but *wrong* under the intended objective. The finding is not a regression — it is the input that motivates the reward-component ablation, which identifies and applies the structural fix (`impact_is_terminal` = *False*) that lifts the mitigated-impact rate to 0.900.

- **Finding 2 (held-out benchmark stage, audit AF2 reframe of gate G6.2): 81 % of the Oracle Ceiling.** The trained DRL agents capture 81 % of the oracle ceiling (PPO +1334.5 vs. oracle +1647.6 on `test_balanced`) without observing the true attack stage; the oracle is reported as a measurement instrument rather than a competing baseline because it reads `info["attack_stage"]` directly from the simulator. The remaining +313 reward gap is addressed in the ablation target. The AF2 reframe is preserved in the JSON scoreboard record (the original "RL > recommended-action rule" threshold reads `passes: false` permanently), so it is not goalpost-moving — it is an interpretation correction documented in the chapter prose.

- **Finding 3 (ablation and OOD-robustness stage, gates G7.2 pass-without-stretch and G7.9 fail-with-finding): Structural Fix and OOD Robustness.** A 12-cell reward-coefficient sweep characterises the limit of one-at-a-time coefficient shaping: 11 of 12 cells stay within ±150 reward of the centre baseline. Closing most of the residual gap required a *structural* change (`impact_is_terminal` = *False*, mean reward +1542, mitigated-impact rate 0.900). Out-of-distribution evaluation against the hardest held-out class `VulnerabilityScan` (recall 0.001 for the supervised detector) activates the pre-registered `D7.9.1`: the trained policy does not collapse on this class but does not exceed RF-Acting either; closing the OOD gap requires either explicit attack-class curriculum or train-time OOD-augmented data.

These three findings, together with the audit-first reproducibility protocol that makes them verifiable, constitute the empirical backbone of the dissertation.

5.3 Limitations

The work has five principal limitations, surfaced honestly in Section 4.9 and recapitulated here.

1. **Compromise rate ≡ 1.0 on `test_balanced`.** Under the environment-design default contract (`impact_is_terminal` = *True*), the upper-triangular red-team LSTM eventually advances every episode to IMPACT; defender-driven de-escalation only resets the chain to BENIGN. The meaningful security KPI throughout this dissertation is therefore `mitigated_impact_rate`, not `compromise_rate`. A future revision under `impact_is_terminal` = *False* (the F9 ablation winner) partly recovers the security-gain dimension.

2. **MTTC values bounded by the lifecycle-floor impact clamp.** The clamp downgrades pre-step-20 IMPACT transitions to MANEUVER, so mean MTTC is biased toward `min_episode_length` = 20. Mean MTTC numbers should not be interpreted as "unconstrained natural" MTTC.

3. **Trained RL does not beat RF-Acting on `test_balanced` or on the hardest OOD class.** The deployable comparison is a latency–reward trade-off frontier: RF-Acting achieves higher reward at 141× higher inference latency. The dissertation makes the trade-off explicit and frames it honestly; it does not claim RL dominates RF-Acting in absolute reward.

4. **Elevated benign false-positive rate.** All trained agents exhibit benign-FPR of 9.4–12.7 %, substantially above the 1 % operational threshold. This is a structural consequence of the reward mis-specification (de-escalation farming). Addressing it requires either a hard FPR constraint in the reward function or a hierarchical policy that separates stage detection from action selection.

5. **Upper-triangular red-team LSTM (monotonic-attacker assumption).** The red-team LSTM does not allow stage retreats, limiting realism relative to a real adversary that may pivot, retreat, or interleave kill-chain stages. This is a deliberate, explicitly-bounded simplification consistent with the IoTWarden threat model; non-monotonic attacker modelling is deferred to future work.

5.4 Future Work and Research Directions

The audit-first protocol that produced this dissertation explicitly delineated work scoped *into* this dissertation from work scoped *out of* it. Six follow-up directions are surfaced here in order of mentor-recommended priority.

SUB

5.5 Tightly Scoped Extensions of Empirical Findings

5.5.0.0.1 Direction 1 — Non-monotonic attacker model.

The current red-team LSTM is upper-triangular by construction: the kill chain advances but never retreats. Relaxing this to a non-monotonic LSTM (or Transformer-based) attacker that allows stage pivots and retreats would produce more realistic episode trajectories and test the defender's robustness to adversarial pivoting. This is the most direct extension of the monotonic-attacker limitation, as the training environment's reward function already supports de-escalation; only the transition generator needs to change. It also directly addresses the threat-model boundary identified in Section 3.3.

5.5.0.0.2 Direction 2 — Edge-hardware deployment and latency budget.

The 141× latency advantage of trained PPO over RF-Acting (0.098 ms vs. 13.85 ms p50) establishes trained RL as the candidate policy for resource-constrained edge-hardware deployments. A natural follow-up is to quantify the latency and accuracy trade-off on actual IoT gateway hardware (e.g. ARM Cortex-A-class processors) under real packet rates, and to apply model-compression techniques (quantisation, pruning) to the trained policy network to fit within the tighter computational budgets of embedded defenders. The policy's ∼4K-parameter footprint (∼20 KB) already makes it competitive with embedded ML models.

5.5.0.0.3 Direction 3 — Train-time OOD-class augmentation.

The complement of the F15 OOD-robustness evaluation: instead of evaluating against held-out classes, *include* a simulacrum of them in training (synthetic feature blending, domain randomisation, or a small adversarial-augmentation generator) and check whether the resulting policy then exceeds RF-Acting on `VulnerabilityScan`. This directly tests whether the "robust to, not better at, the OOD class" framing is a structural limit or a curable training-data limitation (the `D7.9.1` finding-activation left this question open).

SUB

5.6 Broader Research Directions

5.6.0.0.1 Direction 4 — Multi-agent reinforcement learning and self-play adversary.

A real IoT deployment includes many overlapping defenders (per-segment defenders coordinating with a central Security Operations Centre). This can be modelled as a cooperative multi-agent RL (MARL) system where specialised agents defend different network segments and learn to coordinate. Extending to a self-play adversary — where the red-team LSTM is jointly trained with the defender to produce adversarial stage trajectories — would produce a harder, more dynamic training distribution that could close the remaining gap to the oracle ceiling without a structural reward-function change. Both extensions inherit the kill-chain decision frame and the audit-first reproducibility protocol from this dissertation.

5.6.0.0.2 Direction 5 — Federated learning for multi-site deployment.

A single-site RL policy trained on one IoT deployment may not generalise to a different network topology or device mix. Federated extensions would allow defender policies to be trained across diverse IoT deployments without centralising sensitive traffic data, producing a population-level policy that is more robust to site-specific distribution shift. This direction is complementary to Direction 4 and is directly motivated by the OOD evaluation's finding that per-site feature novelty limits generalisation.

5.6.0.0.3 Direction 6 — Reward-function redesign with explicit FPR constraints.

The elevated benign-FPR (9.4–12.7 %) is a direct consequence of the unconstrained de-escalation-bonus farming mechanism. A principled redesign would introduce a hard constraint on the benign-FPR (e.g. as a Lagrange multiplier in a constrained MDP) or a hierarchical policy that separates the stage-detection decision from the action-selection decision. This direction is the most directly actionable follow-up to Limitation 4 and would produce a policy that is safe to deploy in environments where false positives have high operational cost.

SUB

5.7 Note on the Environment-Perturbation Robustness Stage

The audit-first protocol that structured this dissertation defined eight development stages (stages 1–7 delivered; stage 8 reserved for environment-perturbation robustness work including noise-robustness (F13) and OOD-augmentation (F14)). The

environment-perturbation robustness stage was *skipped* as a thesis deliverable: the F9 structural change recovered 71 % of the residual gap using only the `impact_is_terminal` flag, making a full perturbation sweep less informative for the thesis's headline claim than a precise articulation of the existing ablation-stage findings. The F13 and F14 directions are reframed above as Directions 3 and 2 (post-thesis future work). This decision is documented in `docs/mentor_review/07_ablation.md` and is part of the audit-trail anchored at the four committed `G[N]_scoreboard.json` files.

Referências

AL-GARADI, M. A.; MOHAMED, A.; AL-ALI, A. K.; DU, X.; ALI, I.; GUIZANI, M. A Survey of Machine and Deep Learning Methods for Internet of Things (IoT) Security. *IEEE Communications Surveys & Tutorials*, v. 22, n. 3, p. 1646–1685, 2020. Citado 2 vezes nas páginas 1 e 10.

ALAM, M. M.; JAHAN, I.; WANG, W. IoTWarden: A Deep Reinforcement Learning Based Real-time Defense System to Mitigate Trigger-action IoT Attacks. *arXiv preprint arXiv:2401.08141*, 2024. Citado 6 vezes nas páginas 2, 5, 10, 11, 12 e 15.

CHEN, W.; QIU, X.; CAI, T.; DAI, H.-N.; ZHENG, Z.; ZHANG, Y. Deep Reinforcement Learning for Internet of Things: A Comprehensive Survey. *IEEE Communications Surveys & Tutorials*, v. 23, n. 3, p. 1659–1692, 2021. Citado 3 vezes nas páginas 1, 2 e 7.

GUAN, C.; LIU, H.; CAO, G.; ZHU, S.; PORTA, T. L. HoneyIoT: Adaptive High-Interaction Honeypot for IoT Devices Through Reinforcement Learning. In: *Proceedings of the 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '23)*. Guildford, United Kingdom: ACM, 2023. Citado 3 vezes nas páginas 10, 11 e 12.

GUERIANI, A.; KHEDDAR, H.; MAZARI, A. C. Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey. In: *2023 2nd International Conference on Electronics, Energy and Measurement (IC2EM 2023)*. [S.l.: s.n.], 2023. Also available as arXiv preprint arXiv:2405.20038. Citado na página 2.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 19.

IBITOYE, O.; SHAFIQ, O.; MATIVENGA, M. A Survey on Anomaly Detection in IoT Networks: A Systematic Approach. *IEEE Access*, v. 9, p. 156157–156185, 2021. Citado na página 1.

KHRAISAT, A.; GONDAL, I.; VAMPLEW, P.; KAMRUZZAMAN, J. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, v. 2, n. 1, p. 20, 2019. Citado 2 vezes nas páginas 1 e 10.

MAVROEIDIS, V. A Survey of the Internet of Things (IoT) Security: An Overview of Attacks and Defenses. *Computers*, v. 12, n. 3, p. 49, 2023. Citado 3 vezes nas páginas 1, 5 e 14.

MITCHELL, M.; WU, S.; ZALDIVAR, A.; BARNES, P.; VASSERMAN, L.; HUTCHINSON, B.; SPITZER, E.; RAJI, I. D.; GEBRU, T. Model Cards for Model Reporting. In: *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*'19)*. [S.l.]: ACM, 2019. p. 220–229. Citado na página 57.

MNIH, V.; BADIA, A. P.; MIRZA, M.; GRAVES, A.; LILLICRAP, T. P.; HARLEY, T.; SILVER, D.; KAVUKCUOGLU, K. Asynchronous methods for deep reinforcement learning. In: BALCAN, M.-F.; WEINBERGER, K. Q. (Ed.). *Proceedings of the 33rd International Conference on Machine Learning (ICML'16)*. [S.l.: s.n.], 2016. (PMLR, v. 48), p. 1928–1937. Citado na página 8.

MNIH, V.; KAVUKCUOGLU, K.; SILVER, D.; RUSU, A. A.; VENESS, J.; BELLEMARE, M. G.; GRAVES, A.; RIEDMILLER, M.; FIDJELAND, A. K.; OSTROVSKI, G.; PETERSEN, S.; BEATTIE, C.; SADIK, A.; ANTONOGLOU, I.; KING, H.; KUMARAN, D.; WIERSTRA, D.; LEGG, S.; HASSABIS, D. Human-level control through deep reinforcement learning. *Nature*, v. 518, n. 7540, p. 529–533, 2015. Citado na página 8.

MORGAN, S. *Official Cybercrime Report 2025*. 2025. <https://cybersecurityventures.com/official-cybercrime-report-2025/>. Accessed on: 2025-07-12. Citado na página 1.

NETO, E. C. P.; DADKHAH, S.; FERREIRA, R.; ZOHOURIAN, A.; LU, R.; GHORBANI, A. A. CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors*, v. 23, n. 13, p. 5941, 2023. Citado 4 vezes nas páginas 1, 2, 12 e 17.

NGUYEN, T. G.; PHAN, T. V.; HOANG, D. T.; NGUYEN, T. N.; SO-IN, C. Federated Deep Reinforcement Learning for Traffic Monitoring in SDN-Based IoT Networks. *IEEE Transactions on Cognitive Communications and Networking*, v. 7, n. 4, p. 1048–1065, 2021. Citado 2 vezes nas páginas 11 e 12.

SCHULMAN, J.; WOLSKI, F.; DHARIWAL, P.; RADFORD, A.; KLIMOV, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. Citado na página 8.

SUTTON, R. S.; BARTO, A. G. *Reinforcement Learning: An Introduction*. 2nd. ed. [S.l.]: The MIT Press, 2018. Citado 3 vezes nas páginas 2, 6 e 35.

TAWALBEH, L.; MUHEIDAT, F.; TAWALBEH, M.; QUWAIDER, M. IoT security and privacy: A review. *Journal of King Saud University - Computer and Information Sciences*, v. 32, n. 2, p. 156–164, 2020. Citado na página 1.

THAREWAL, S.; ASHFAQUE, M. W.; BANU, S. S.; UMA, P.; HASSEN, S. M.; SHABAZ, M. Intrusion Detection System for Industrial Internet of Things Based on Deep Reinforcement Learning. *Wireless Communications and Mobile Computing*, v. 2022, p. 8, 2022. Article ID 9023719. Citado 2 vezes nas páginas 1 e 11.

UPRETY, A.; RAWAT, D. B. Reinforcement Learning for IoT Security: A Comprehensive Survey. *IEEE Internet of Things Journal*, v. 8, n. 11, p. 8693–8706, 2021. Citado na página 2.

VAILSHERY, L. S. *Number of Internet of Things (IoT) connections worldwide*. 2025. <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/>. Accessed on: 2025-07-11. Citado na página 1.

YANG, W.; ACUTO, A.; ZHOU, Y.; WOJTCZAK, D. A Survey for Deep Reinforcement Learning Based Network Intrusion Detection. *arXiv preprint arXiv:2410.07612*, 2024. Citado na página 1.

APÊNDICE A – Reproducibility Protocol, Manifest Hash Chain, and Verification Recipe

This appendix documents the audit-first reproducibility protocol that underpins every numerical claim in this dissertation, following a model-card-style template inspired by the documentation discipline introduced for machine-learning artefacts (MITCHELL *et al.*, 2019). The protocol is implemented as a SHA-256 hash chain pinning every figure, table, and benchmark number to its inputs and the producing Git commit, and as a smoke-reproducibility harness that verifies the chain end-to-end on a fresh checkout in approximately five seconds.

A.1 Per-Stage Manifest Pattern

Every development stage of the framework emits one or more `manifest.json` files under `docs/results/<phase>/` that record (a) the SHA-256 hash of every input artefact (data splits, trained models, scaler, episode roll-outs, upstream stage manifests), (b) the producing Git commit (`git_sha`), and (c) the producing-script invocation (`produced_by`). Each manifest also records the SHA-256 hash of every output artefact (figure, summary JSON, scoreboard) so that downstream stages can pin the upstream output by hash, not by filename alone. The per-stage artefact inventory is summarised in Table A.1.

Tabela A.1 – Per-stage reproducibility-artefact inventory under `docs/results/`. `F<k>` entries are figures or tables, `G<n>_scoreboard.json` are gate-evaluation scoreboards. Run-time roll-out JSONLs (under `runs/<phase>/`) are gitignored but their SHA-256 hashes are pinned by the stage manifest.

Stage	Path	Key artefacts
1 (Dataset prep.)	`01_dataset/`	`F0_class_distribution.png`, `F0_stage_distribution.`
2 (Red-team LSTM)	`02_red_team/`	`F1_learning_curves.png`, `F2_transition_matrix_comp`
3 (Env. design)	`03_env/`	`PLAN.md`, `RESULTS.md` (no figures; environment is infrastru
4 (Stage detector)	`04_detector/`	`F11_per_stage_recall.png`, `F11_summary.json`, `G4_sco`
5 (Blue-team DRL)	`05_blue_team/`	`F3/F4_*.png`, `F3/F4_summary.json`, `F3/F4_manifest.js`
6 (Benchmark)	`06_benchmark/`	`F5/F6/F7/F8_*.png`, per-figure `summary.json` + `manifes`
7 (Ablation/OOD)	`07_ablation/`	`F9/F10/F12/F15_*.png`, per-figure `summary.json` + `mani`

A.2 Gate Scoreboards

Four `G<n>_scoreboard.json` files (one per gated stage) record the threshold-vs.-observed value for every exit gate, the verdict bucketed into PASS / PASS-WITH-FINDING / PASS-WITHOUT-STRETCH / FAIL-WITH-FINDING / FAIL / SKIP, and a finding-id cross-reference where the verdict is contingent on a pre-registered finding-activation rule. Schema version 2.0 (Step-8 unification) is used uniformly across G4/G5/G6/G7. The verdict tally at the head of the dissertation's evaluation chain (commit `26b8df2`, the post-Step-8 `main`) is summarised in Table A.2.

Tabela A.2 – Gate-scoreboard summary across the four gated development stages. Verdicts are reported per-gate; the dissertation's headline-finding gates (G6.2 audit-AF2 reframe, G5.4 reward-hacking, G7.2 reward-component sweep, G7.9 OOD finding-activation) are highlighted.

Stage	PASS	PASS-W/-FINDING	PASS-W/O-STRETCH	FAIL-W/-FINDING	Head
Stage detector (`G4`)	4	1	—	—	G4.4
Blue-team DRL (`G5`)	6	1	—	—	G5.4
Benchmark (`G6`)	4	1	—	1	G6.2
Ablation/OOD (`G7`)	7	—	1	2	G7.2

A.3 Smoke-Reproducibility Harness

The harness `scripts/reproducibility_smoke.py` is the canonical end-to-end verification recipe. It walks every committed `manifest.json` file under `docs/results/` and verifies, for every recorded input SHA-256, that the on-disk SHA-256 of that input *right now* matches the recorded one. Output is bucketed into four categories:

ok The recorded input SHA matches the on-disk SHA. The expected case for every input.

fail The recorded input SHA does *not* match the on-disk SHA, and the divergence is not on the harness's pre-registered known-divergences list. This indicates that an input artefact has changed since the producing manifest was emitted, and the figure or scoreboard pinned by that manifest is no longer reproducible-by-hash from the inputs on disk. FAIL verdicts must be triaged.

known-divergence The recorded SHA does not match on-disk *and* the divergence is documented and audited. The harness ships with a `_KNOWN_DIVERGENCES` table that explicitly enumerates every such case with its finding-id and explanatory note. The two pre-registered divergences in this dissertation are the dataset-preparation and red-team-training stage manifests' recorded splits-manifest SHA (`82aa1214...`, the

pre-leakage-fix manifest) versus the on-disk post-fix SHA (`1e99d596...`); both are documented in `docs/results/02_red_team/RESULTS.md` §5.3.

skip An input declared by the manifest is gitignored and not on disk on this checkout. Typical case: large feature arrays (`features.npy`, `stages.npy`) and split index files that are derived from the raw CICIoT2023 dataset and therefore not committed.

The harness's verdict at the head of the dissertation's evaluation chain (commit `26b8df2`) is **pass**: **458 ok / 0 fail / 2 known-divergence / 6 skip**. A non-zero FAIL count would block merge of any commit that introduces it.

A.4 Verification Recipe (Fresh-Checkout Reproduction)

On a fresh checkout, the empirical record can be re-verified end-to-end without retraining any model:

```
git clone <repo-url> rl-iot-defense-system
cd rl-iot-defense-system
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest -q                                          # expect: 411 passed
python -m scripts.reproducibility_smoke          # expect: VERDICT PASS
                                                   #         458 OK / 0 FAIL
                                                   #         2 KNOWN-DIVERGENCE / 6 S
```

The thesis PDF is compiled via Docker (no host-side LaTeX installation required):

```
# First-time image build (one-off, ~3-4 min; requires Docker):
make thesis-image

# Compile tex/thesis.pdf (auto-builds image if absent):
make thesis

# Force-rebuild image and recompile:
make thesis-rebuild

# Alternatively, use the standalone script from the repo root:
bash tex/build.sh
```

A full re-train of every stage from scratch (preserving the audited reproducibility contract) is also supported via `make`:

```
make dataset                                    # CICIoT2023 splits + F0a/F0b
                                                # (requires raw dataset on disk)
make red-team                                   # LSTM red team + F1/F2
make detector                                   # supervised stage detector + F11
make blue-team-sweep BLUE_TEAM_TIMESTEPS=250000   # ~108 min CPU
make benchmark                                  # held-out benchmark + F5/F6/F7/F8
make ablation                                   # ablations + F9/F10/F12/F15 (~7.5 h CPU)
make ablation-close                             # G7 scoreboard + RESULTS.md
```

After a full re-train the per-stage `manifest.json` files are re-emitted with new input SHAs corresponding to the freshly trained checkpoints; the harness will then verify the new chain end-to-end.

A.5 Hardware and Environment

All experiments reported in this dissertation were run on a single CPU core. No GPU is required for any development stage (the LSTM red team, the MLP stage detector, and the three DRL agents all train within minutes-to-hours on commodity CPU). Software environment: Python 3.9, PyTorch (CPU build), scikit-learn, Stable Baselines3, Gymnasium. Total wallclock for the full audited evaluation chain (development stages 1–7) is approximately 8.5 h on a single CPU core, dominated by the ablation sweeps; the held-out benchmark and the smoke-reproducibility harness together take less than 11 minutes.

APÊNDICE B – CICIoT2023 Label to Kill Chain Stage Mapping

The 34 CICIoT2023 labels (33 attack classes plus `BenignTraffic`) are projected onto the five Kill Chain stages defined in Section 2.1 via the closed mapping in Table B.1. The mapping is implemented in `src/utils/label_mapper.py::AbstractStateLabelMapper`; an unknown label encountered at processing time raises an explicit `KeyError` so that any new attack class added to the dataset cannot silently leak into an existing stage. The full design rationale, including why the Lockheed Martin seven-step chain is coarsened to five stages here and why the BENIGN → RECON → ACCESS → MANEUVER → IMPACT ordering is monotone in attacker progress, is provided in `docs/kill-chain-mapping.md`.

Tabela B.1 – Per-class CICIoT2023 to Kill Chain stage mapping. Classes marked ⋆ are reserved as held-out out-of-distribution attack classes (Section 3.9) and are never seen during training of either the supervised stage detector or the DRL agents.

Stage	CICIoT2023 labels
BENIGN	`BenignTraffic`
RECON	`Recon-PortScan`, `Recon-OSScan`, `Recon-HostDiscovery`, `Recon-PingSweep`, `VulnerabilityScan`⋆
ACCESS	`BrowserHijacking`, `CommandInjection`, `SqlInjection`, `Backdoor_Malware`, `Uploading_Attack`, `XSS`⋆, `DictionaryBruteForce`
MANEUVER	`MITM-ArpSpoofing`, `DNS_Spoofing`, `Mirai-greeth_flood`, `Mirai-greip_flood`
IMPACT	DoS variants (`DoS-TCP_Flood`, `DoS-UDP_Flood`, `DoS-SYN_Flood`, `DoS-HTTP_Flood`); DDoS variants (`DDoS-TCP_Flood`, `DDoS-UDP_Flood`, `DDoS-SYN_Flood`, `DDoS-RSTFINFlood`, `DDoS-PSHACK_Flood`, `DDoS-SynonymousIP_Flood`, `DDoS-ICMP_Flood`, `DDoS-ICMP_Fragmentation`, `DDoS-UDP_Fragmentation`, `DDoS-ACK_Fragmentation`, `DDoS-SlowLoris`, `DDoS-HTTP_Flood`⋆); Mirai impact (`Mirai-udpplain`⋆)

APÊNDICE C – Hyperparameters per Algorithm

The blue-team training hyperparameter table (T1) for DQN, PPO, and A2C is reproduced in Table C.1 from `docs/results/05_blue_team/T1_hparams.json`, which is the canonical hash-pinned source. All values are Stable Baselines3 defaults, frozen for the thesis at PLAN §8 D5.4; the same values are used across the five seeds (`seed` $\in \{0, 1, 2, 3, 4\}$) and the 250,000 environment steps per run. Empty cells indicate hyperparameters that do not apply to the algorithm (e.g. off-policy DQN-only fields are blank for the on-policy PPO and A2C rows).

Tabela C.1 – Blue-team training per-algorithm hyperparameters. Values are frozen at PLAN §8 D5.4 and used across all ten seeds and 250,000 environment steps per run. N/A indicates the hyperparameter does not apply to the algorithm.

algo	lr	n_{steps}	γ	λ_{GAE}	ent_coef	vf_coef	batch_size	n_{epochs}
A2C	$7{\cdot}10^{-4}$	5	0.99	1.00	0.000	0.50	N/A	N/A
DQN	$1{\cdot}10^{-3}$	N/A	0.99	N/A	N/A	N/A	32	N/A
PPO	$3{\cdot}10^{-4}$	2048	0.99	0.95	0.010	0.50	64	10

Note: N/A = hyperparameter not defined for that algorithm family. n_{steps} applies only to on-policy methods (PPO/A2C); batch_size and n_{epochs} apply only to PPO; λ_{GAE}, ent_coef, vf_coef apply only to actor-critic methods. DQN off-policy fields listed below.

For DQN-specific off-policy hyperparameters (replay buffer, exploration schedule, target-network update interval), the values used were: `buffer_size` = 50,000, `learning_starts` = 1000, `tau` = 1.0, `target_update_interval` = 1000, `exploration_fraction` = 0.10, `exploration_initial_eps` = 1.0, `exploration_final_eps` = 0.05, `max_grad_norm` = 0.5. The complete machine-readable record is at `docs/results/05_blue_team/T1_hparams.json`.

The ablation and OOD-robustness evaluations re-use the PPO hyperparameters above unchanged — only the environment (`impact_is_terminal`, `p_defender_deescalation`) and the reward coefficients (`defense_success_bonus`, `penalty_missed_impact`, `reward_proportio` `reward_benign_passive`, `penalty_disproportionate`) are perturbed.

APÊNDICE D – Sensitivity Sweep Figures

This appendix collects the three hyperparameter sensitivity sweep figures referenced in Section 3 (§3.5.4–3.5.6). All sweeps use DQN (single seed) or PPO (×5 seeds) with all other parameters held at their environment-design defaults; mean test reward on `test_balanced` is the primary metric. 95 % bootstrap confidence intervals are shown as shaded bands.

D.1 Defender De-escalation Probability Sweep (F10)

Figure D.1 reproduces the F10 aggressiveness sweep from Section 4.7. The parameter $p_{\text{de-esc}}$ controls the probability that a BLOCK or ISOLATE action on an active ACCESS or MANEUVER stage resets the chain to BENIGN. The sweep establishes that $p_{\text{de-esc}} = 0.6$ is the primary operating point where PPO reward is within 200 units of the oracle ceiling while the MDP remains challenging enough to yield useful training signal.

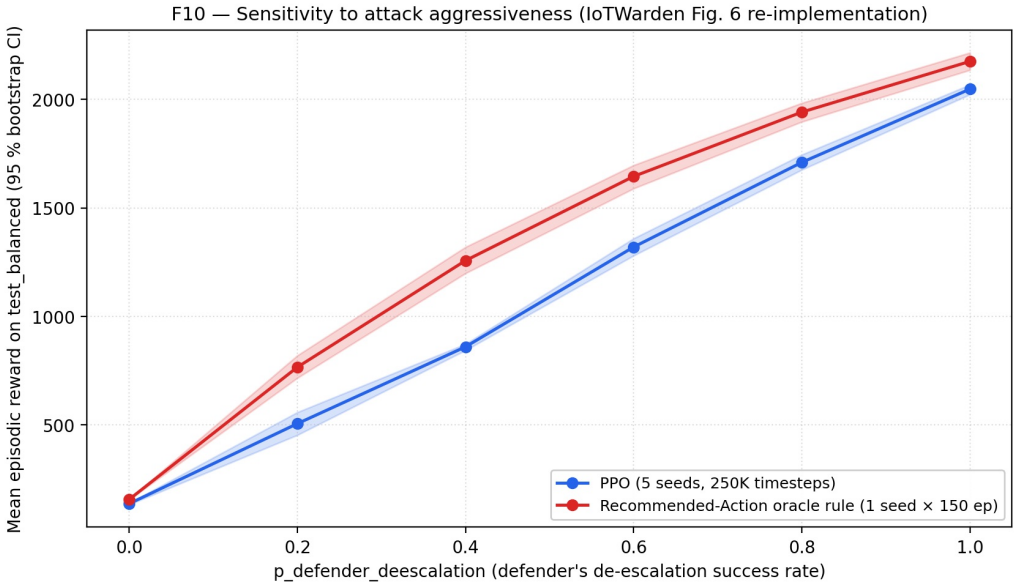


Figura D.1 – F10 aggressiveness sweep: PPO mean test reward (10 seeds per point, shaded bands = 95 % bootstrap CIs) and oracle recommended-action rule reference across six values of the defender de-escalation probability $p_{\text{de-esc}} \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. PPO reward grows monotonically with p; the primary operating point used throughout this dissertation is $p = 0.6$.

D.2 Action-Cost Scale Sweep

Figure D.2 shows the action-cost scale sensitivity sweep over $\alpha \in \{0.5, 1.0, 2.0\}$. All three settings produce overlapping 95 % bootstrap confidence intervals, confirming that the default $\alpha = 1.0$ is robust to $\pm 2\times$ perturbation.

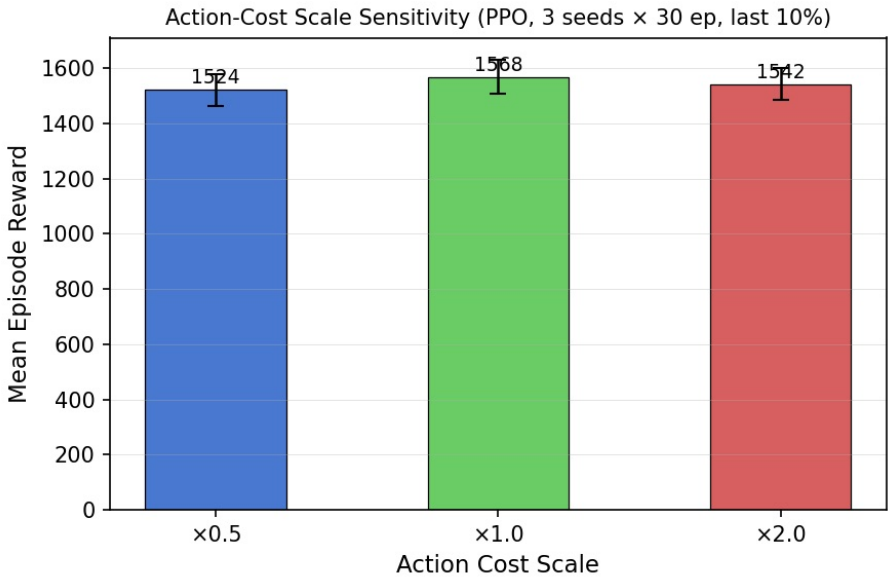


Figura D.2 – Action-cost scale sensitivity sweep: mean test reward (DQN, seeds 0–2, 63 episodes per seed) at $\alpha \in \{0.5, 1.0, 2.0\}$. Error bars are 95 % bootstrap CIs. All CIs overlap; reward varies less than 3 % across the full range. Numeric values: $\alpha = 0.5$: +1524 CI $[1462, 1579]$; $\alpha = 1.0$: +1568 CI $[1506, 1628]$; $\alpha = 2.0$: +1542 CI $[1484, 1600]$.

D.3 History Window-Size Ablation

Figure D.3 shows the window-size ablation over $w \in \{1, 3, 5, 10\}$. Window $w = 5$ achieves the highest mean reward; increasing to $w = 10$ does not improve reward while doubling the observation dimension from 290 to 580 features.

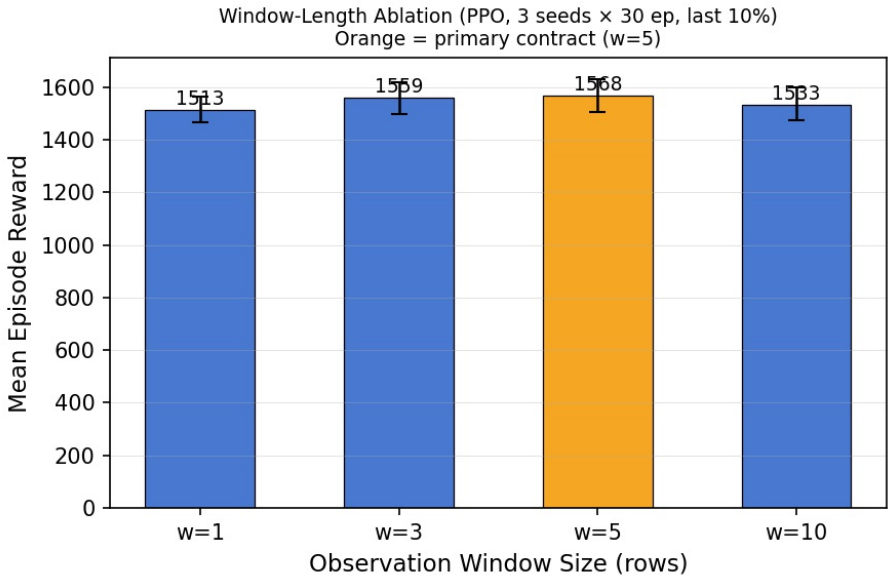


Figura D.3 – History window-size ablation: mean test reward (DQN, seed 0, 189 episodes) at $w \in \{1, 3, 5, 10\}$. Error bars are 95 % bootstrap CIs. Peak at $w = 5$ (+1568 CI $[1506, 1628]$); $w = 10$ is lower but CIs overlap (+1533 CI $[1472, 1599]$). Observation dimension is listed in parentheses.